BaitFL: Lightweight Trap-Based Free-Rider Detection and Mitigation in Federated Learning

Aashnan Rahman
Department of Computer Science and Engineering
Islamic University of Technology
MSc Thesis

***Abstract*—Federated learning relies on participants performing useful local optimization, but free riders can consume the shared model while submitting unchanged or fabricated responses. Detecting such behavior is difficult when honest clients have heterogeneous data and naturally different update statistics. This paper investigates BaitFL, a server-side challenge protocol that compares responses to undisclosed perturbations of the same model. Robust update-norm and tensor-profile measurements first nominate anomalous clients; separate geometric separation tests and repeated fresh evidence authorize removal. Component-specific memory retains strong evidence across ambiguous cycles, while bounded quarantine prevents weak suspicion from causing indefinite exclusion. The method requires no auxiliary detector training, client-reported loss, or attack-specific history reconstruction. We evaluate BaitFL across multiple random seeds on MNIST and CIFAR-10, covering four fabrication strategies, IID and Dirichlet non-IID data, and 30% and 40% free riders over 100 training rounds. The method achieves 100.00% recall and pooled precision of 99.91%. Most executions complete removal at round 30, while difficult cases require up to 90 rounds. There are 3 permanent honest-client removals, and 32 honest client-cases experience temporary quarantine. Matched comparisons with unprotected federated averaging improve mean final accuracy by 0.10 percentage points on MNIST and 0.64 on CIFAR-10. In a paired MNIST timing run, BaitFL's mean round computation time is 29.35% lower than that of a STD-DAGMM implementation using five fitting epochs per round. Adaptive adversaries and secure aggregation remain outside the study's scope.**

***Index Terms*—federated learning, free-rider detection, hidden challenges, robust statistics, non-IID data, client auditing**

I. Introduction

Federated learning (FL) allows multiple clients to train a shared model without handing over their local data. In each communication round, the server distributes the current global model, the clients update it using their own data, and the server aggregates the returned updates into a revised model [1]. Since raw data stays with its owner, FL is well suited to domains such as healthcare and personal mobile services, where pooling sensitive records in one place is often impractical or undesirable [1], [2]. This protection is not absolute, as shared updates can still leak information [3]. Exchanging updates instead of datasets also avoids moving large volumes of local data, although repeated model exchange remains a significant communication cost in FL [1]. Effective collaboration depends on clients performing useful local training, even though the server cannot directly observe that work.

Like other distributed systems, FL is open to misconduct. Label-flipping and model-poisoning attacks, for example, alter local data or submitted updates in order to degrade the global model or introduce targeted behavior [3], [4]. Free riding is a quieter problem. The participant's primary aim is to obtain the shared model without paying the cost of local training [5], [6]. A client may lack useful data or computational resources, or may simply choose to conserve them. Instead of training, it can return an unchanged model, fabricate a noisy response, or build an update from models received in earlier rounds. Such responses may resemble what the server expects, so they need not visibly disrupt learning and can evade checks aimed at overtly harmful updates. Free riding shifts the training burden to honest participants. When a client withholds useful data and computation, the federation also loses a contribution that could have improved the shared model.

Detecting this omission is difficult because unusual updates are not limited to free riders. Honest clients differ in sample sizes, class distributions, and local learning trajectories, particularly under non-IID data [3]. A small update can therefore reflect legitimate training, while a fabricated one can look like an ordinary contribution. The server has to tell these cases apart without observing local optimization. Existing approaches rely on learned anomaly models, parameter audits, or evidence obtained through inference attacks [5], [7], [8]. This raises a practical question: can lightweight server-side checks detect free riding without training an auxiliary detector? There is also a cost to being wrong. Withholding an honest client's updates can reduce its contribution to the shared model, even if it is never permanently removed. A detector must therefore consider both the evidence needed for removal and the duration of temporary exclusion.

To address this problem, we propose *BaitFL*, a lightweight trap-based method for detecting and mitigating free riders in federated learning.

The main contributions of this work are as follows:

- **Lightweight server-side detection.** We develop a challenge-based detector using update norms and tensor profiles, without training an auxiliary detection model or accessing clients' private training data.
- **Confirmation using repeated evidence.** We design a decision mechanism that separates initial suspicion from permanent removal and requires fresh evidence of the

same component across two challenges and distinct audit cycles for statistical confirmation.
- **Bounded mitigation and rehabilitation.** We retain strong evidence across ambiguous cycles while allowing unconfirmed quarantine to expire and clearing statistical memory after repeated clean observations.
- **Evaluation across four free-rider behaviors.** We study unchanged-model responses, random fabrication, history averaging, and trajectory imitation on MNIST and CIFAR-10 under IID and non-IID partitions, with 30% and 40% free riders.

The remainder of the paper reviews related work, defines the threat model and the free-rider behaviors evaluated, describes the auditing method in detail, and reports the resulting detection performance, computational cost, and limitations.

II. Literature Review

A. Free-Rider Attacks and Evasion

Lin *et al.* studied clients that fabricate updates without local training data, including random submissions and responses derived from changes in the global model [5]. They showed that taking part in model exchange does not establish taking part in training. The evaluated fabrications were detectable, but this does not guarantee that every non-training client will remain distinguishable from honest clients.

Fraboni *et al.* analyzed free riding under iterative model aggregation and developed stochastic disguises that follow the trajectory of the shared model [6]. Their analysis showed that a free rider can approach the model obtained by contributing participants, which makes simple unchanged-model checks insufficient. Zhu *et al.* analyzed local and global training dynamics with an Ornstein–Uhlenbeck model and proposed scaled-delta and advanced free-rider attacks that address both convergence and stealth [9]. These attacks expose a weakness of passive update screening, since a fabricated update can be designed to match the statistics a detector observes. This motivates auditing responses to controlled challenges instead of relying only on how ordinary updates look.

B. Learned Detection and Contribution Auditing

To detect fabricated updates, Lin *et al.* proposed STD-DAGMM, which combines an autoencoder-based representation, reconstruction information, and update standard deviation with mixture-based anomaly scoring [5]. Adding the standard deviation improved separation for attacks that the original DAGMM detector struggled with. This gives richer evidence than a single scalar statistic, but it requires fitting an auxiliary encoder, decoder, and estimation network on high-dimensional updates. Training and storing this additional model adds computation and memory cost that compact update summaries avoid. Section V-H reports the fitting cost measured in our STD-DAGMM implementation.

Wang *et al.* proposed FRAD for AIoT, which combines DAGMM with contribution and reputation information [10]. Contribution is modeled from computing resources, communication cost, and data quality, and a PageRank-based reputation guides participant selection. This extends anomaly detection with historical evidence, and the authors report detection under high free-rider proportions. FRAD still fits an auxiliary detector, however, and it needs resource and quality signals beyond the returned model updates, which are an extra deployment requirement to obtain reliably. Contribution ranking alone also cannot tell whether a low-ranked client skipped training.

Separately, Wang *et al.* introduced PASS, in which clients audit other participants' updates on their own private datasets and return accuracy-based contribution scores [7]. PASS addresses both anonymous and selfish free riders, including evaluated settings where attackers are the majority. Peer auditing, however, adds local model evaluation, update distribution, and score reporting. It also gives clients access to individual updates, a privacy exposure that PASS mitigates with Gaussian perturbation and parameter pruning, at the price of reduced audit utility. The authors leave network-cost validation open, since their standalone simulator does not measure communication. In addition, an accuracy-based score depends on the evaluator's data, so under heterogeneity low utility and absence of training need not coincide.

C. Canary-Based Evidence and Privacy Tradeoffs

Recasens *et al.* proposed FRIDA, which repurposes membership and property inference to obtain evidence that a client trained [8]. Its membership-based variants ask clients to train on server-provided canary examples and assess the effect through loss or gradient similarity. This creates an explicit training signal rather than relying on incidental update statistics. Canary construction and distribution, though, require additional labeled examples, canary training adds client work beyond the original local task, and loss or gradient testing adds server computation. The signal also shows that the canary task was used, not that the client's private data contributed useful work.

FRIDA's property-based variants infer characteristics such as local label distributions and test their consistency or diversity [8]. They avoid canary training but deliberately extract information about local data, which may itself be sensitive. The two families therefore carry different costs: extra training on challenge data for membership tests, and disclosure of inferred dataset properties for property tests.

Secure aggregation, developed by Bonawitz *et al.*, combines client updates without revealing them individually [11]. This protects individual contributions but conflicts with detectors that need attributable responses. BaitFL shares this limitation. Avoiding raw-data access and property inference does not make individual model responses private, and the method cannot be inserted unchanged into standard secure aggregation.

Taken together, these limitations point to three practical requirements: less auxiliary detector training, no additional client-side evaluation or canary tasks, and a clear separation between suspicion and sufficient evidence for removal. BaitFL studies model perturbations and repeated evidence from update norms and tensor profiles under these requirements. Its challenge rounds still consume client work and withhold responses

from aggregation, so execution cost and honest-client exclusion must be evaluated alongside detection quality.

III. Threat Model

A. Participants and Server Knowledge

We consider a trusted server coordinating N clients with persistent identities and reliable communication. All active clients respond in every round, with no client sampling, late arrivals, or dropouts. Removed clients no longer participate. Quarantined clients continue to respond. Their responses and all responses to traps are excluded from aggregation. Honest clients train the model they receive using the prescribed optimizer. Their local data may differ in size and class distribution, and the experiments cover both IID and non-IID partitions.

The server can send different model states to different clients. Clients are not told whether the model they receive is an ordinary global model or a trap. For client i in round t, the server knows the model it sent, $w_{t,i}^{\text{send}}$, and the model returned by the client, $w_{t,i}^{\text{ret}}$. It computes the update and its norm as

$$\Delta_{t,i} = w_{t,i}^{\text{ret}} - w_{t,i}^{\text{send}}, \qquad r_{t,i} = \|\Delta_{t,i}\|_2. \tag{1}$$

Screening uses the returned models and the server's audit history. It does not require private training records or prior knowledge of which clients are free riders or how they fabricate their responses. The server observes individual client responses, so this setting does not use secure aggregation.

B. Free-Rider Objective and Scope

The free riders considered here seek access to the shared model without performing local training. A client may have no suitable data, lack the resources needed to train the model, or possess both but choose not to use them. It can still receive and store models and perform the relatively simple calculations needed to fabricate a response. Its objective is to avoid training costs rather than to introduce a targeted error or backdoor into the global model.

We study persistent free riding: a free rider performs no local optimization at any stage of an experiment. Clients that train for fewer local steps or alternate between honest training and free riding are outside this scope. Each free rider uses only its own received-model history and local randomness. Free riders do not collude, share responses, or change identities. Their response strategies do not adapt to identify or evade audits. Adaptive strategies and the possibility of recognizing traps are discussed in Section VIII-C.

We assume an honest majority because the detector uses robust peer statistics as its reference and seeks a separated minority of suspicious responses. The experiments use 30% and 40% free riders. Each experiment evaluates one fixed fabrication strategy for all free riders. The four strategies are defined below.

C. Evaluated Free-Rider Behaviors

We denote the model returned by a free rider as $\widehat{w}_{t,i}$.

FR1: current-model sendback. The client returns the received model unchanged,

$$\widehat{w}_{t,i} = w_{t,i}^{\text{send}}. \tag{2}$$

This produces a zero update without local training, regardless of whether the received model is a trap.

FR2: bounded random fabrication. The client adds independent uniform noise to each floating point entry of the received model state,

$$\widehat{w}_{t,i} = w_{t,i}^{\text{send}} + \epsilon_{t,i}, \quad (\epsilon_{t,i})_j \sim \mathcal{U}(-10^{-3}, 10^{-3}). \tag{3}$$

The added noise makes the response differ from the received model without performing any local training.

FR3: received-history averaging. Let H be the history length and $\mathcal{H}_{t,i}$ contain the most recent $\min(H, t)$ model states received by client i. The client returns their mean,

$$\widehat{w}_{t,i} = \frac{1}{|\mathcal{H}_{t,i}|} \sum_{w \in \mathcal{H}_{t,i}} w. \tag{4}$$

The history includes the current model and any traps received during earlier rounds. Averaging these states produces a response based on the shared model's recent evolution without using local training data.

FR4: received-trajectory imitation. The client computes differences between consecutive received models,

$$d_{s,i} = w_{s,i}^{\text{send}} - w_{s-1,i}^{\text{send}}. \tag{5}$$

Let $\overline{d}_{t,i}$ be the mean of up to five most recent differences. The response is

$$\widehat{w}_{t,i} = w_{t,i}^{\text{send}} + \overline{d}_{t,i} + \eta_{t,i}. \tag{6}$$

Once differences are available, the client draws a standard Gaussian vector $z_{t,i}$ and rescales it to obtain

$$\eta_{t,i} = 0.1 \, \frac{\max(\|\overline{d}_{t,i}\|_2, \epsilon_0)}{\max(\|z_{t,i}\|_2, \epsilon_0)} \, z_{t,i}, \qquad \epsilon_0 = 10^{-6}. \tag{7}$$

These vectors contain all floating model entries. For its first response, $\overline{d}_{1,i} = 0$ and $\eta_{1,i}$ instead has independent Gaussian coordinates with standard deviation 10^{-4}. Received traps enter the difference history just like ordinary models, so they also affect the estimated trajectory. This strategy follows recent model changes while adding variation to the fabricated response.

IV. Proposed Methodology

BaitFL audits clients by observing how they respond to perturbed models. The procedure has four phases: warmup, initial paired coverage, confirmation and review, and sparse auditing. Update statistics identify clients for further checks. Permanent removal requires repeated evidence, while temporary exclusion has a separate review budget. The client records $\mathcal{S}$ store each client's status, evidence, and review progress.

A. Challenges, Measurements, and Aggregation

At the start of a paired audit cycle, the server takes a copy $w^{(c)}$ of the current global model and creates two traps, A and B. For floating tensor k, trap $a \in \{A, B\}$ is

$$\tau_{a,k} = w_k^{(c)} + \gamma \max\{\mathrm{sd}(w_k^{(c)}), \epsilon_\tau\}\xi_{a,k}, \quad \xi_{a,k} \sim \mathcal{N}(0, I). \tag{8}$$

The two traps use independent noise. We set $\gamma = 0.01$ and $\epsilon_\tau = 10^{-4}$. Perturbations apply to floating model tensors, including BatchNorm buffers. Integer counters are unchanged. Both traps remain fixed for the cycle, even as ordinary updates change the global model.

For each response, the server records the update norm r_i and a tensor profile p_i. Its coordinate for tensor k is

$$p_{i,k} = \frac{\|\Delta_{i,k}\|_2}{\sum_j \|\Delta_{i,j}\|_2}. \tag{9}$$

When the denominator is below 10^{-6}, the profile is set to zero. The norm measures update magnitude, while the profile describes how that magnitude is distributed across model tensors.

Clients are compared only with others that received the same trap. The reference for client i consists of all other active clients with a response to that trap. Let m_i be the median of their update norms and d_i the median absolute deviation. The normalized norm score is

$$z_i^n = \frac{r_i - m_i}{d_i/\Phi^{-1}(0.75)}, \tag{10}$$

where Φ is the standard normal cumulative distribution function. For the profile score, the server first forms the coordinatewise median of the peer profiles. It measures each profile's Euclidean distance from that reference and normalizes the client's distance using the median and median absolute deviation of the peer distances. This gives z_i^p. A zero median absolute deviation leaves the corresponding score undefined.

Only ordinary responses from eligible clients enter aggregation. Let $\mathcal{A}_t$ be the clients that were neither trapped nor quarantined during round t, after excluding any clients removed or newly quarantined at its end. The aggregation routine used in the algorithms is

$$w_{t+1} = w_t + \frac{1}{|\mathcal{A}_t|} \sum_{i \in \mathcal{A}_t} \Delta_{t,i}. \tag{11}$$

If $\mathcal{A}_t$ is empty, the model is unchanged. Clients receive equal weight. Training on a trap replaces ordinary local training for that round.

B. Phase I: Warmup

The first ten rounds use the ordinary global model. Clients return their responses and the server aggregates them, as shown in Algorithm 1, lines 3 to 6. This gives the federation an initial training trajectory before audits begin. Client evidence counters start at zero, and no client is removed during warmup.

Algorithm 1 Phase I: ordinary training during warmup

```
1: procedure WARMUP(w, S)
2:     for t = 1, ..., 10 do
3:         Send w to all active clients
4:         Collect responses and record their update summaries
5:         A_t ← all active clients
6:         w ← AGGREGATE(w, A_t)
7:     end for
8:     return (w, 10, S)
9: end procedure
```

C. Phase II: Initial Paired Coverage

Rounds 11 to 20 give every client one response to each trap. The server shuffles clients into at most ten groups. It assigns pairs of groups to traps during the first half of the cycle, then repeats the assignments with the opposite trap. With 100 clients, approximately 20% receive a trap in each round. The remaining clients receive the current global model.

Algorithm 2 describes this collection procedure and its reuse in later confirmation cycles. The server chooses one response position for each client and trap before observing the responses, as shown on line 11. It waits for the complete pair before making a cycle decision on lines 21 to 22. The last round's aggregation follows that decision so newly excluded responses do not enter the global model.

1) Screening: The server calculates $D_i = z_i^p - z_i^n$ for trap A, trap B, and their average. For each view, it builds a histogram with $\lceil\sqrt{n_v}\rceil$ equally spaced bins, where n_v is the number of clients with valid paired scores. An Otsu split provides a boundary between the two score groups [12]. The split is accepted only when the lower side of the averaged view contains a majority of active clients. The server then uses 1,000 bootstrap samples to estimate a 99% interval for each boundary [13]. The same resampled clients are used in all three views.

A client above the upper boundary in every view is nominated as suspicious. One below the lower boundary in every view is clear. The remaining clients are candidates for further checking. If a usable split cannot be obtained, norms below the peer median on both traps nominate suspicion. Neither norm below the median gives a clear result. A single low norm or an undefined score gives a candidate result.

2) Geometric evidence: Suspicion must pass an additional separation test before it counts toward removal. On each trap, let $\mathcal{C}$ be the nominated cohort and $\mathcal{P}$ the remaining clients. With n active clients, the cohort must satisfy

$$\left\lceil \frac{n}{\lceil\sqrt{n}\rceil} \right\rceil \le |\mathcal{C}| < |\mathcal{P}|. \tag{12}$$

Quartiles are calculated separately for norms and profile distances. Write $Q_q(\mathcal{C})$ for the empirical quantile at level q and $I_C = Q_{.75}(\mathcal{C}) - Q_{.25}(\mathcal{C})$ for its interquartile range. For low update norms, evidence is accepted when

$$\begin{aligned} Q_{.25}(\mathcal{P}) - Q_{.75}(\mathcal{C}) &> I_C, \\ \min(\mathcal{P}) - Q_{.75}(\mathcal{C}) &> I_C. \end{aligned} \tag{13}$$

Algorithm 2 Phase II: collect a complete pair of trap responses

Require: Training horizon T
procedure PAIREDCYCLE($w, t, \mathcal{S}$, initial)
  Advance the cycle number c
  Create τ_A, τ_B from w using Eq. (8)
  if initial **then**
    Shuffle clients into groups and schedule paired coverage
  else
    Separate clients under review from ordinary clients
    Schedule alternating traps for reviewed clients
    Give ordinary clients one response to each trap
  end if
  Preselect one position per client and trap
  Initialize selected observations O
  $L \leftarrow \min(10, T - t)$
  for $s = 1, \ldots, L$ **do**
    $t \leftarrow t + 1$
    Read the scheduled trap and recipient set $\mathcal{T}_t$
    Send the trap to $\mathcal{T}_t$ and w to other clients
    Collect responses and store the preselected observations
    $\mathcal{A}_t \leftarrow$ ordinary clients not quarantined at round start
    if $s = 10$ **then**
      $E \leftarrow$ SCREENANDCERTIFY(O)
      $\mathcal{S} \leftarrow$ CONFIRM($\mathcal{S}, E, c$)
      Exclude new removals and quarantines from $\mathcal{A}_t$
    end if
    $w \leftarrow$ AGGREGATE($w, \mathcal{A}_t$)
  end for
  Update the audit clock from the remaining review state
  return $(w, t, \mathcal{S})$
end procedure

A nominated client must also have a norm below the median of $\mathcal{P}$. These conditions require separation from the peer population, rather than concentration within the nominated group alone.

For high profile distances, define $I_P = Q_{.75}(\mathcal{P}) - Q_{.25}(\mathcal{P})$ and $g = Q_{.25}(\mathcal{C}) - Q_{.75}(\mathcal{P})$. Evidence requires $g > I_C$, $g > 0$, and either

$$Q_{.25}(\mathcal{C}) - \max(\mathcal{P}) > I_C \quad \text{or} \quad \frac{I_C}{|\mathcal{C}| - 1} < \frac{I_P}{|\mathcal{P}| - 1}. \tag{14}$$

The client's distance must exceed the median of $\mathcal{P}$. A norm or profile certificate is recorded only when the same component supports that client on both traps. Exact copies of the current or immediately preceding sent model are recorded separately on each trap.

After these checks, S denotes strong paired evidence, C denotes an unresolved candidate, and R denotes a clear cycle result. Only a paired geometric certificate contributes to statistical confirmation. The routine SCREENANDCERTIFY in Algorithm 2 applies these rules and returns the component certificates, exact replay flags, and cycle outcomes.

D. Phase III: Confirmation and Bounded Review

The server keeps separate certificate counts $M_{i,n}$ and $M_{i,p}$ for each client. A fresh paired certificate increases its own count once in a completed cycle. Ambiguous cycles retain the counts. Two consecutive clear cycles erase them. Statistical removal requires a fresh certificate whose component count reaches two, as shown in Algorithm 3, lines 7 to 19. Norm evidence in one cycle and profile evidence in another do not confirm one another. Exact replay uses a separate rule that requires evidence on both traps in two consecutive completed cycles. With the ten warmup rounds and ten rounds per paired cycle, the earliest removal is at round 30.

A fresh strong result places the client in quarantine and starts a review budget of two cycles. Another fresh strong result renews that budget. Otherwise, it decreases after each completed cycle. When the budget expires, the client is released even if unresolved evidence remains in memory. A candidate without strong evidence may contribute on ordinary rounds. Two consecutive clear results rehabilitate a suspected or candidate client. One additional clear cycle completes probation before routine auditing resumes. Clients released from unresolved review have a cooldown of two sparse audits before weak evidence can enroll them again. Fresh paired evidence can still start review.

During confirmation, reviewed and ordinary clients are grouped separately. At most ten groups are allocated in proportion to the two populations. Reviewed clients receive alternating traps throughout the cycle. Ordinary clients receive each trap once. As in the initial cycle, the decision uses only the response positions selected in advance. Each new cycle creates fresh traps. Algorithm 2 collects these responses, and Algorithm 3 updates the client records at the cycle boundary.

E. Phase IV: Sparse Audits

When no client remains under review or probation, the server returns to ordinary training and schedules a sparse audit after ten rounds. In an audit round, all active clients receive the same fresh trap, so the global model remains unchanged for that round. Ordinary rounds between audits use Eq. (11).

The audit applies the same norm and profile measurements to one trap. Screening nominates a cohort, and the geometric tests identify clients for further review. Exact replay also triggers review. Algorithm 4, lines 5 to 10, shows this step. Eligible clients enter a two cycle review without immediate quarantine. The server then resumes paired cycles. A sparse audit cannot increase statistical certificate counts, clear certificate memory, or authorize removal.

F. Overall Procedure

Algorithm 5 connects the four phases. The client records $\mathcal{S}$ contain membership, quarantine and review status, certificate counts, clean progress, probation, and cooldowns. After warmup, every client receives initial paired coverage. Further paired cycles run while review or probation is pending. Otherwise, the server uses ordinary training with sparse audits until the training horizon is reached.

Algorithm 3 Phase III: confirm evidence and manage review

procedure CONFIRM($\mathcal{S}, E, c$)
  for all active clients i **do**
    $J_i \leftarrow$ components certified on both traps in E
    Update the consecutive paired exact replay count
    if $J_i \neq \emptyset$ **then**
      for all $j \in J_i$ **do**
        $M_{i,j} \leftarrow M_{i,j} + 1$
      end for
      Reset the consecutive clear cycle count
    else if the cycle outcome is R **then**
      Advance the consecutive clear cycle count
      if two consecutive clear cycles are complete **then**
        $M_{i,n} \leftarrow 0$, $M_{i,p} \leftarrow 0$
      end if
    else
      Reset the clear cycle count and retain certificate memory
    end if
    if a fresh component count reaches two or exact replay is confirmed **then**
      Permanently remove i
    else if $J_i \neq \emptyset$ or paired exact replay is present **then**
      Quarantine i and set its review budget to two cycles
      Reset its clean progress, probation, and cooldown
    else
      Decrease the review budget toward zero
      Apply clear cycle rehabilitation and probation
      if unconfirmed review has just expired **then**
        Release i and set a cooldown of two sparse audits
      end if
    end if
  end for
  return $\mathcal{S}$
end procedure

Algorithm 4 Phase IV: ordinary training and sparse auditing

procedure SPARSEROUND($w, t, \mathcal{S}$)
  $t \leftarrow t + 1$
  if a sparse audit is due **then**
    Create a fresh trap and send it to all active clients
    Collect responses and compute norm and profile scores
    $\mathcal{F} \leftarrow$ clients selected by geometric evidence or exact replay
    Identify clients currently in audit cooldown
    Decrease audit cooldowns toward zero
    for all $i \in \mathcal{F}$ not in the identified cooldown set **do**
      Open two cycles of review without quarantine
      Reset clean progress and consecutive exact replay evidence
    end for
    Update the audit clock from the screening outcome
  else
    Send w to all active clients and collect responses
    $\mathcal{A}_t \leftarrow$ ordinary clients not quarantined
    $w \leftarrow$ AGGREGATE($w, \mathcal{A}_t$)
  end if
  return $(w, t, \mathcal{S})$
end procedure

Algorithm 5 Overall BaitFL procedure

Require: Initial model w, clients, training horizon $T \geq 30$
Initialize client records $\mathcal{S}$ and zero evidence counts
$(w, t, \mathcal{S}) \leftarrow$ WARMUP($w, \mathcal{S}$)
initial $\leftarrow$ true
while $t < T$ and active clients remain **do**
  if initial or review or probation is pending **then**
    $(w, t, \mathcal{S}) \leftarrow$ PAIREDCYCLE($w, t, \mathcal{S}$, initial)
    initial $\leftarrow$ false
  else
    $(w, t, \mathcal{S}) \leftarrow$ SPARSEROUND($w, t, \mathcal{S}$)
  end if
end while
return w and the final client records $\mathcal{S}$

V. Results

A. Experimental Setup

We evaluate BaitFL on MNIST and CIFAR-10, which contain handwritten digit images and color images from ten object classes, respectively [14], [15]. The evaluation covers four free-rider behaviors, IID and non-IID client data, and free-rider proportions of 30% and 40%. Each configuration is repeated using multiple random seeds, and Table I reports the corresponding mean performance. The optimizer and detector settings remain fixed across replications.

Each federation contains 100 clients and trains for 100 communication rounds. For IID experiments, data are randomly divided into client partitions of equal size. For non-IID experiments, class proportions are allocated using a Dirichlet distribution with $\alpha = 0.5$, with allocation repeated if a client receives no samples. Replications vary data partitioning, attacker assignment, model initialization, training randomness, and audit scheduling.

Honest clients perform three local SGD epochs per round with learning rate 0.01, momentum 0.9, and batch size 32. FR3 uses history length $H = 5$. The MNIST model has two convolutional blocks with 16 and 32 channels and a 128-unit hidden classifier. The CIFAR-10 model has three convolutional blocks with BatchNorm and 32, 64, and 128 channels, followed by a 256-unit hidden classifier. Images are normalized without data augmentation, and all experiments use CPU execution.

For utility and execution-time comparisons, we use completed runs of federated averaging without a defense and a measured STD-DAGMM implementation. The matched baseline settings are described in Section V-G, and the detector timing experiment in Section V-H.

B. Evaluation Metrics

A positive detection is *permanent removal*, not nomination or quarantine. For true positives TP, honest removals FP, and missed free riders FN,

$$P = \frac{TP}{TP + FP}, \qquad R = \frac{TP}{TP + FN}, \qquad F_1 = \frac{2TP}{2TP + FP + FN}. \tag{15}$$

Precision is undefined if no client is removed. Complete-detection round t_{all} exists only when every free rider has been removed; otherwise it is NR. Final global test accuracy is a separate utility metric. We additionally report final honest-candidate counts, honest clients ever quarantined, quarantine client-rounds, trap client-rounds, and all excluded active honest client-rounds. These last categories can overlap and must not be summed as disjoint costs.

C. Permanent Detection and Model Utility

Table I groups the results by dataset, partition, and free-rider proportion. Precision and recall are calculated over the four attack types within each seed and then averaged across seeds. Final test accuracy is averaged separately for FR1 to FR4, while round time averages all attack types and seeds with equal weight. Round time includes local training, aggregation, global evaluation, and detection.

Every free rider is removed in the evaluated replications. Pooled over all replications, precision is 99.91%, recall is 100.00%, and F_1 is 99.96%. The three honest removals occur in MNIST non-IID at 40% free riders, with two in an FR3 execution and one in an FR4 execution. Complete detection occurs between rounds 30 and 90. Most executions finish at round 30, while some MNIST FR4 and non-IID CIFAR-10 FR3 or FR4 executions require additional audit cycles.

Mean final accuracy ranges from 98.83% to 99.02% on MNIST and from 74.51% to 77.84% on CIFAR-10. The score diagnostics below examine a representative replication. The comparison with unprotected federated averaging uses matched executions and is reported separately in Table IV and Figure 5.

TABLE I
FREE-RIDER DETECTION PERFORMANCE AND GLOBAL MODEL UTILITY.

Dataset	Partition	FR (%)	Detection (%)		Final test accuracy (%)				Round time (s)
			Precision	Recall	FR1	FR2	FR3	FR4	
MNIST	IID	30	100.00	100.00	99.01	99.01	99.02	98.90	42.38
		40	100.00	100.00	98.93	98.94	98.92	98.83	37.05
	Non-IID	30	100.00	100.00	98.92	98.93	98.93	98.91	41.62
		40	99.39	100.00	98.91	98.91	98.90	98.88	36.08
CIFAR-10	IID	30	100.00	100.00	77.84	77.84	77.76	77.77	99.35
		40	100.00	100.00	77.07	77.13	76.92	76.97	85.35
	Non-IID	30	100.00	100.00	75.94	75.86	75.77	75.78	92.30
		40	100.00	100.00	74.70	74.88	74.67	74.51	77.03

D. Representative Score Geometry and Evidence Evolution

Figures 1 and 2 show round-20 client response scores on MNIST and CIFAR-10. Each figure contains 16 panels. The four columns correspond to FR1 through FR4. From top to bottom, the rows show 30% IID, 30% non-IID, 40% IID, and 40% non-IID settings. Blue circles represent honest clients and orange triangles represent free riders. Each point averages one client's norm and profile scores over the two challenges. The displayed coordinates are $\operatorname{asinh}(\bar{z}_{\mathrm{norm}})$ and $\operatorname{asinh}(\bar{z}_{\mathrm{profile}})$, which preserve ordering while compressing extreme values. Panel axes follow their observed ranges.

FR1 and FR2 generally form distinct response groups, whereas the separation of history-based responses depends more strongly on the dataset and partition. Overlapping points can represent several clients, particularly for exact sendbacks. Colors use evaluation labels after scoring. The server evaluates the two challenges separately, so the displayed average does not replace the paired confirmation rule.

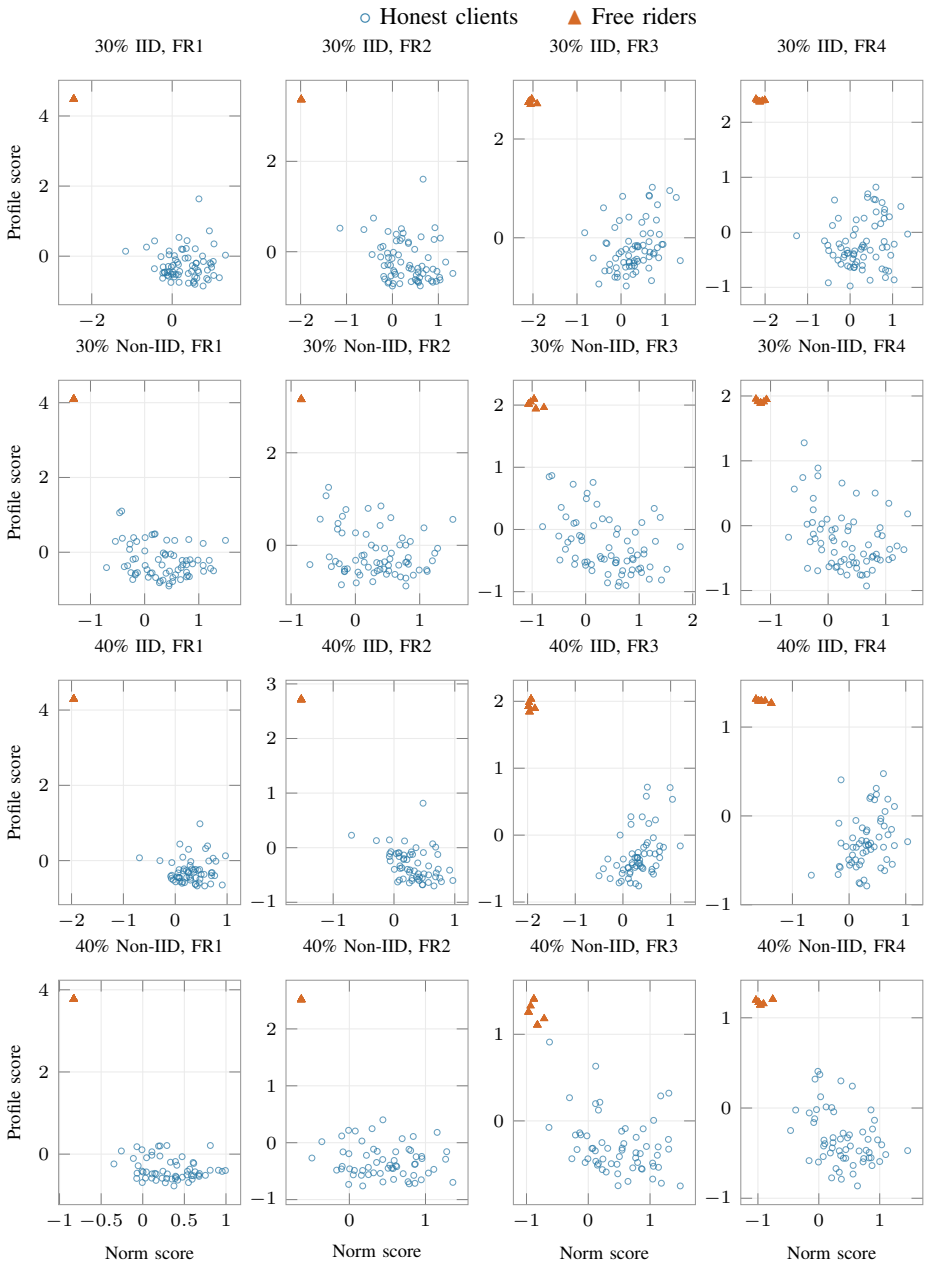


Fig. 1. MNIST client response scores.

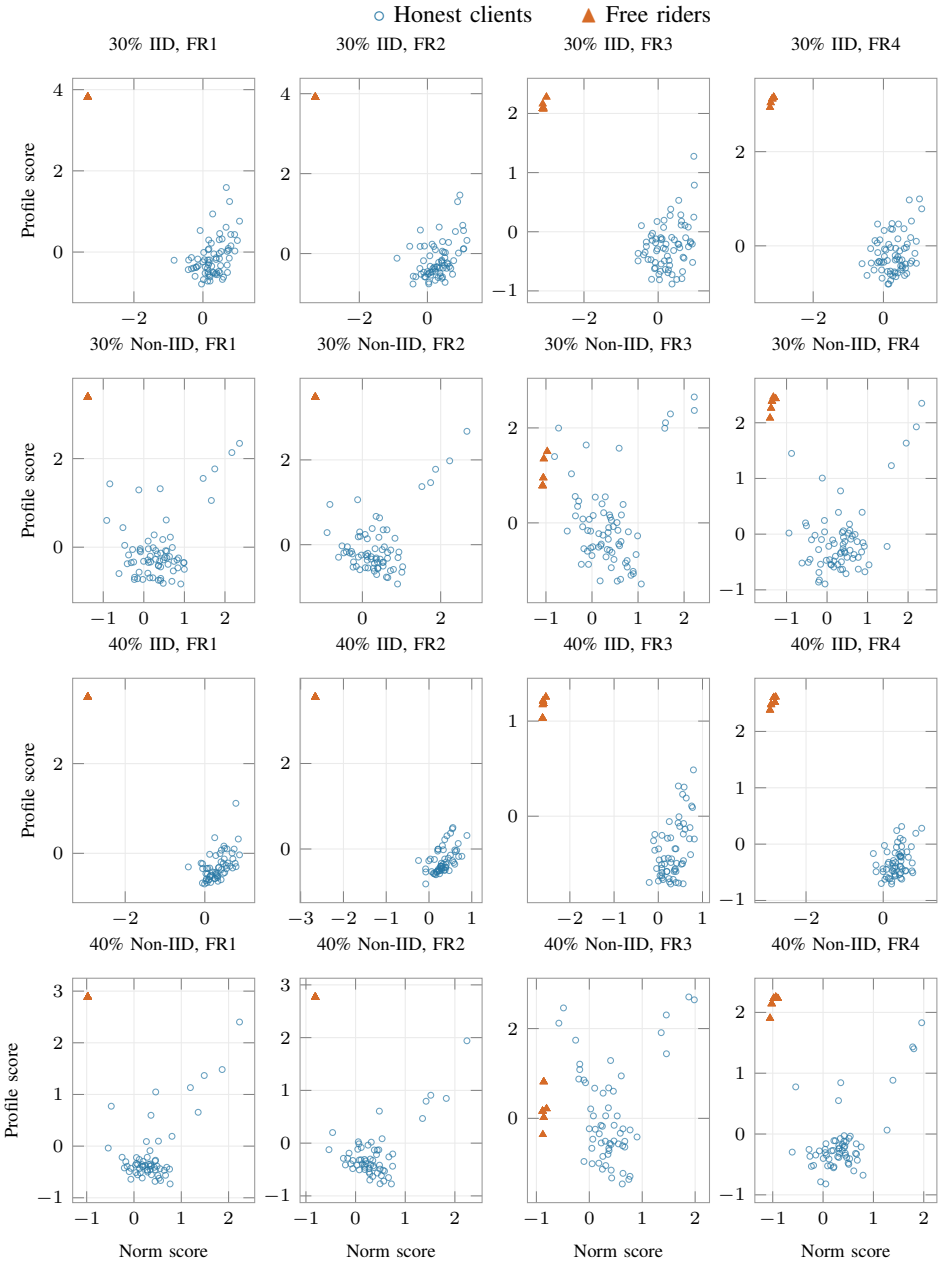


Fig. 2. CIFAR-10 client response scores.

Figure 3 follows two CIFAR-10 non-IID FR3 executions through successive evidence cycles. The top row shows 40% free riders, and the bottom row shows 30%. Blue bars count honest clients and orange outlines count free riders. All panels use the same axis limits. Within each panel, both populations share bins derived from their combined trap-A update norms. The counts below each panel come from the decision logs and require a fresh norm certificate on both challenges. They therefore cannot be read from a single histogram or inferred from a visible gap alone.

At 40%, all 40 free riders and nine honest clients receive their first paired norm certificate at round 20. No client receives fresh norm evidence at round 30, so that cycle adds no norm confirmation. The server retains the earlier evidence. At round 60, all 40 free riders receive a fresh matching certificate and are removed, while none of the nine honest clients receives that second norm certificate. At 30%, the first norm certificates for all 30 free riders arrive at round 40 and are confirmed at round 80. One honest client receives a norm certificate at round 80, but it is insufficient for removal without earlier matching evidence. The diagram between the rows summarizes this distinction between initial suspicion and confirmed removal.

Figure 4 supplements the raw distributions with class mean norm-z and profile-z over complete cycles, with interquartile client dispersion. Class labels are used only after scoring to describe observed behavior; the server never calculates an "honest-only" boundary. A lower free-rider mean alone is insufficient: the strict certificate must still establish cohort support and separation from the peer range on both traps. Because reference populations and MAD scales change after removal, z-score trajectories are not fixed-unit measurements of model improvement. Undefined scores remain missing. The first ten warmup rounds do not provide these paired trap scores, and no warmup score trajectories are fabricated.

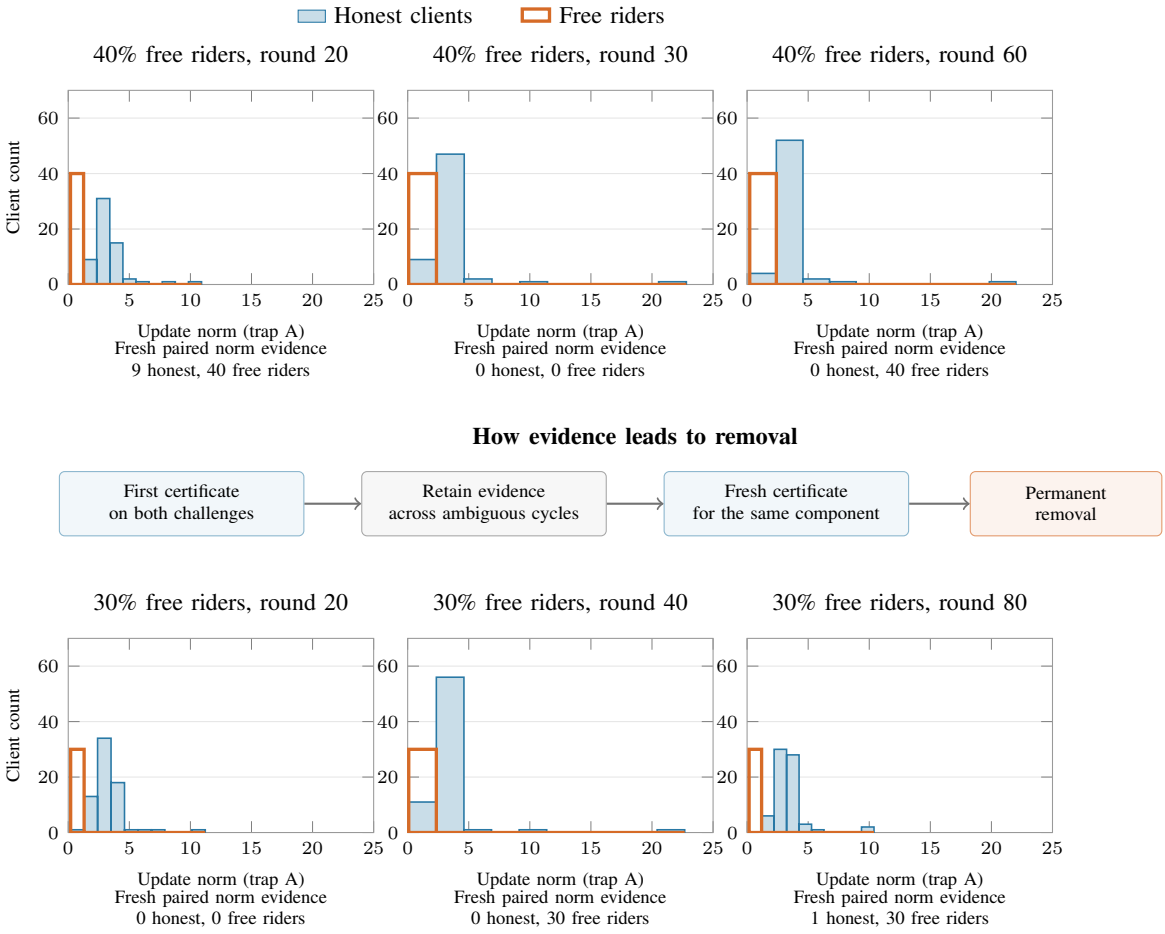


Fig. 3. Update norm evolution and repeated confirmation in CIFAR-10 non-IID FR3.

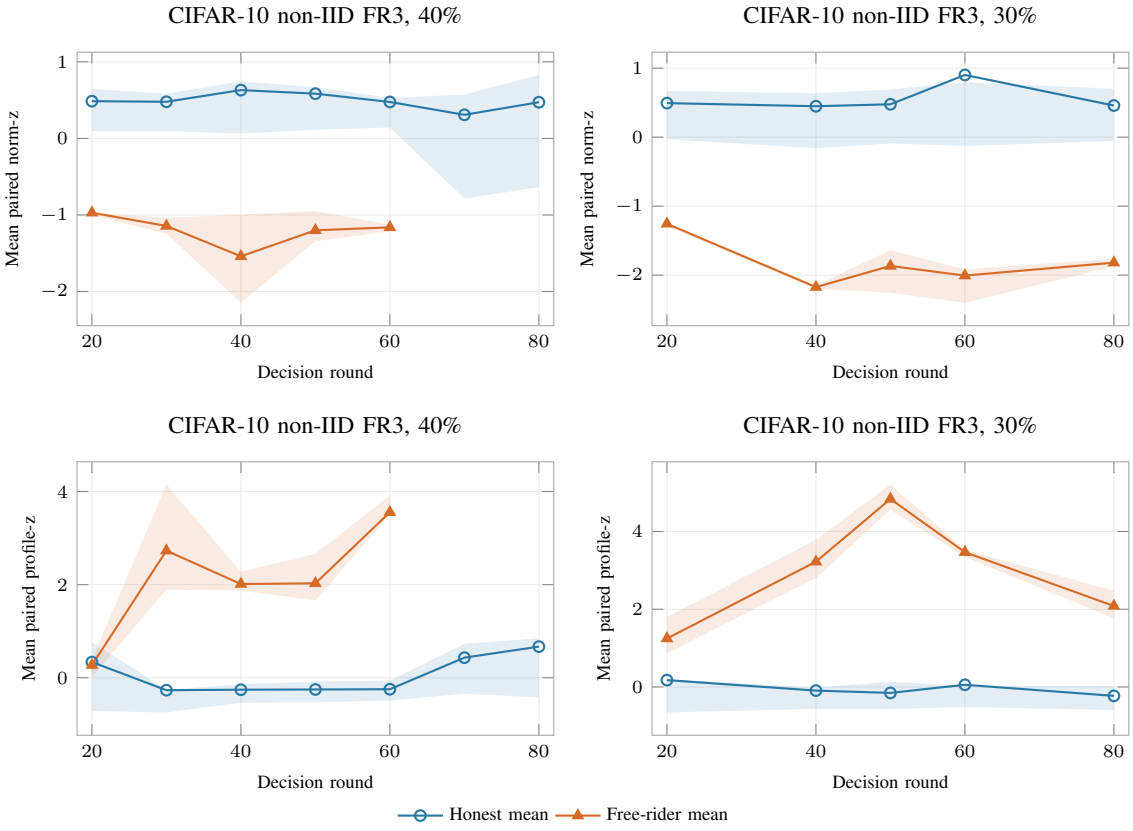


Fig. 4. Evolution of paired response scores.

E. Honest-Client Impact

Across the repeated experiments, 3 honest client-cases are permanently removed and 32 experience quarantine, totalling 620 honest quarantine client rounds. No client remains quarantined at completion. Table II reports means across seeds for every FR3 configuration and for the FR4 configurations in which honest clients experience quarantine or removal. Each count is averaged across seeds, so fractional values describe variation between executions rather than fractions of individual clients.

The honest-client columns distinguish unresolved candidates at completion, permanent removals, and distinct clients ever quarantined. The client-round columns separate quarantine duration, deliberate trap exclusion, and withheld responses outside trap rounds. Quarantine and trap counts may overlap. The final column averages the per-execution percentage of active honest responses withheld from aggregation, counting each response once. The honest population is 70 clients at 30% free riders and 60 at 40%.

Weak candidate labels persist after free riders have been removed. For example, non-IID FR3 at 40% ends with a mean of 37 honest candidates on MNIST and 43 on CIFAR-10. Candidates are not excluded from ordinary aggregation solely because of this label. Their counts are reported separately because initial screening and permanent-removal decisions have different consequences.

F. Execution Cost

Table III reports means across all four attacks and seeds, with the minimum and maximum execution values in brackets. Execution duration in the last column is the mean number of minutes per run. MNIST averages 36.08 to 37.05 seconds per round at 40% free riders and 41.62 to 42.38 seconds at 30%. CIFAR-10 averages 77.03 to 85.35 and 92.30 to 99.35 seconds, respectively. The measurements include serial client simulation, local training, aggregation, and evaluation. The logged round timer ends before checkpoint/log publication; reported execution duration additionally includes initialization, publication work, and recovery work represented by the resumed timer. Powered-off waiting is not a detector computation cost.

Process RSS includes training tensors, simulated attacker histories, and server state. It therefore measures the complete process. The measured baseline comparison below reports changes in execution time, while the operation counts in Section VII describe detector work.

G. Comparison with Federated Averaging without a Defense

We compare BaitFL with ordinary federated averaging in which all 100 clients participate throughout training and every returned response is aggregated. The baseline applies no traps, screening, quarantine, or removal. The main comparison uses the complete common replication across both datasets, both partitions, both free-rider proportions, and all four attacks. Each execution is paired with the same BaitFL configuration and random seed. Training and attack settings agree, and the first ten global accuracies match before challenge rounds begin. Three additional completed CIFAR-10 non-IID executions at 40% provide a further replication check in Table XII.

Table IV averages the four attacks within each dataset, partition, and proportion. These matched values are separate from the multiple-seed means in Table I. Accuracy changes are percentage-point differences between BaitFL and the baseline. Round-time changes are $100(\bar{t}_{\mathrm{BaitFL}}/\bar{t}_{\mathrm{base}} - 1)$, so a positive value indicates a longer round. Final accuracy improves in 24 matched conditions, remains unchanged in one, and decreases in seven. The average gain is 0.10 percentage points on MNIST and 0.64 on CIFAR-10. The largest gain is 2.34 percentage points for CIFAR-10 non-IID FR3 at 40%.

Figure 5 shows rounds 60 to 100. Each panel averages the eight matched attack and partition combinations for one dataset and free-rider proportion. Shading shows their interquartile range. It describes variation across configurations rather than uncertainty across random seeds. Figure 11 provides the same comparison over all 100 rounds. As a measure of late-round consistency, we calculate the standard deviation of accuracy within each execution's final 20 rounds and then average these values. It decreases from 0.034 to 0.028 percentage points on MNIST and from 0.244 to 0.213 on CIFAR-10.

TABLE II
HONEST-CLIENT SCREENING AND EXCLUSION.

Dataset	Partition	FR (%)	Attack	Honest clients			Excluded client rounds			All excluded (%)
				Candidates	Removed	Quarantined	Quarantine	Trap	Non-trap	
MNIST	IID	30	FR3	6.33	0.00	0.00	0.00	770.00	0.00	11.00
		40	FR3	4.33	0.00	0.00	0.00	660.00	0.00	11.00
	Non-IID	30	FR3	41.67	0.00	1.33	26.67	861.33	1.00	12.32
		40	FR3	37.00	0.67	3.33	60.00	762.67	2.67	12.86
			FR4	35.00	0.33	1.00	20.00	872.67	0.67	14.57
CIFAR-10	IID	30	FR3	2.33	0.00	0.00	0.00	770.00	0.00	11.00
		40	FR3	4.00	0.00	0.00	0.00	660.00	0.00	11.00
	Non-IID	30	FR3	42.67	0.00	0.67	13.33	1051.33	0.00	15.02
			FR4	27.33	0.00	0.67	13.33	850.67	0.67	12.16
		40	FR3	43.00	0.00	3.67	73.33	1022.67	2.00	17.08

TABLE III
CPU EXECUTION TIME AND PROCESS MEMORY.

Dataset	Partition	FR (%)	s/round: mean [range]	Peak RSS GiB: mean [range]	min/run
MNIST	IID	30	42.38 [39.78–48.57]	0.54 [0.29–0.78]	75.02
		40	37.05 [35.35–42.68]	0.60 [0.30–0.88]	66.71
	Non-IID	30	41.62 [40.57–43.43]	0.57 [0.30–0.78]	73.79
		40	36.08 [34.34–38.67]	0.62 [0.30–0.88]	65.27
CIFAR-10	IID	30	99.35 [88.18–119.07]	1.24 [0.70–1.76]	175.07
		40	85.35 [73.52–104.21]	1.38 [0.71–2.06]	154.65
	Non-IID	30	92.30 [82.85–108.38]	1.31 [0.73–1.79]	161.97
		40	77.03 [69.10–84.85]	1.49 [0.75–2.14]	139.40

Figure 6 retains the individual attack results. Its upper panels show accuracy changes and its lower panels show round-time changes. The four colors distinguish attacker proportion and partition, and the zero line denotes equal performance. Grouped round time decreases by 1.64% to 8.08% on MNIST and increases by 2.70% to 12.18% on CIFAR-10. These changes include response handling, aggregation, evaluation, and detection along the resulting trajectories. Host load and different timer coverage also affect the comparison, so the percentages do not isolate detector overhead.

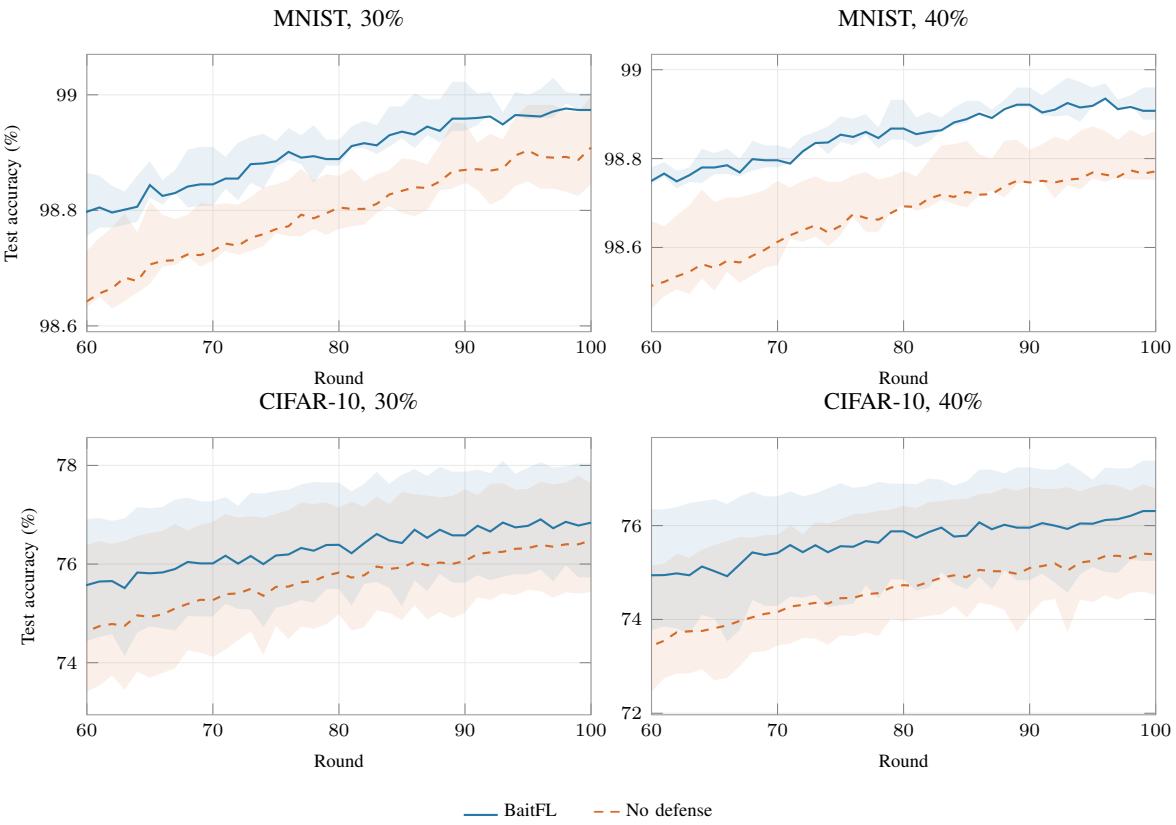


Fig. 5. Late-training accuracy with and without BaitFL.

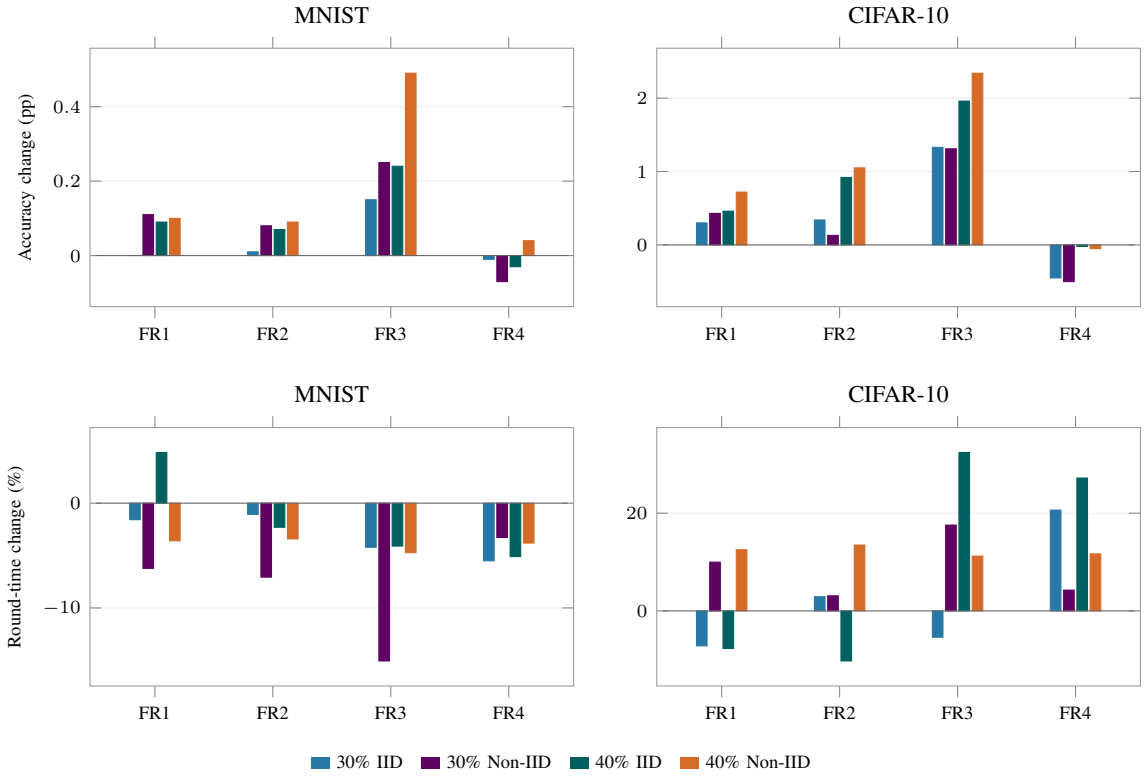


Fig. 6. Changes relative to federated averaging without a defense.

TABLE IV
COMPARISON WITH FEDERATED AVERAGING WITHOUT A DEFENSE.

Dataset	Partition	FR (%)	Final accuracy (%)			Mean round time (s)		
			No defense	BaitFL	Δ (pp)	No defense	BaitFL	Δ (%)
MNIST	IID	30	98.92	98.96	0.04	44.37	42.99	-3.10
		40	98.77	98.86	0.09	37.04	36.43	-1.64
	Non-IID	30	98.90	98.99	0.09	44.91	41.28	-8.08
		40	98.78	98.96	0.18	35.97	34.57	-3.89
CIFAR-10	IID	30	77.67	78.05	0.38	98.57	101.23	2.70
		40	76.61	77.44	0.83	82.83	90.66	9.46
	Non-IID	30	75.29	75.63	0.34	89.30	97.15	8.79
		40	74.17	75.19	1.02	70.84	79.46	12.18

H. Timing Comparison with STD-DAGMM

The timing comparison uses non-IID FR3 at 40%, with 100 clients, 100 rounds, three local SGD epochs, and CPU execution. Paired instrumented MNIST runs use six PyTorch threads and record individual operations as well as complete elapsed rounds. The BaitFL instrumentation preserves the study's training and detection code and reproduces its accuracy trajectory. These timing runs are reported separately from the multiple-seed detection results. STD-DAGMM follows the published mechanism [5], with a selected budget of five fitting epochs per round. All 100 clients remain in its workflow, while BaitFL follows its challenge and removal procedure.

Table V reports MNIST stage means and sample standard deviations over the 100 measured rounds. The upper panels of Figure 7 show these stages side by side. BaitFL's trap construction and scheduling average 1.77 ms per round. Response features and model preparation require 197.90 ms, and decision work requires 15.54 ms. Together, these timers account for 215.21 ms per round. The decision timer covers the screening, audit, and state-transition block, including removal when triggered. Evaluation of fabricated responses is a simulator diagnostic and does not supply detector evidence.

STD-DAGMM fitting averages 15.76 seconds per round. Its joint network training and mixture finalization average 12.64 and 0.39 seconds, respectively, and are included within that fitting time. Preparation, fitting, scoring, and flag decisions together require 16.14 seconds per round. BaitFL has no auxiliary detector-fitting stage. CIFAR-10 retains its recorded BaitFL phase times and STD-DAGMM stage measurements in the lower panels of Figure 7. Its STD-DAGMM fitting and total detector times average 45.03 and 46.04 seconds per round.

Table VII distinguishes warmup, trap-response rounds, screening rounds without removal, removal rounds, single audits, and ordinary rounds. A removal round is counted separately from other screening rounds, and a single audit does not count as a complete paired cycle. The categories cover all 100 rounds. MNIST warmup averages 38.03 seconds of computation, trap-response rounds 37.40 seconds, screening rounds 37.46 seconds, and the removal round 37.52 seconds. The decision block averages 373.03 ms in screening rounds and takes 395.91 ms in the removal round. These are operation times within the corresponding rounds. The final column weights each phase by its frequency to obtain its contribution to the full-run computation mean.

Figure 8 and Table VI separate round computation from elapsed round time. Computation adds the exclusive training, response, challenge, feature, decision, aggregation, and evaluation timers. Elapsed time also includes research logging, checkpoints, and unassigned or harness work. For MNIST, mean computation is 36.92 seconds for BaitFL and 52.26 seconds for STD-DAGMM, a difference of 15.34 seconds or 29.35%. Mean elapsed time is 40.92 and 62.83 seconds, respectively, a difference of 21.91 seconds or 34.87%. All averages use $\bar{t} = \sum_{r=1}^{100} t_r/100$. CIFAR-10 retains its logged computation comparison of 84.85 and 127.24 seconds per round. Its separate BaitFL operation and elapsed-round measurements are not available, as indicated by dashes in the tables.

On MNIST, research logging and checkpoints average 3.99 seconds per round for BaitFL and 3.75 seconds for STD-DAGMM. STD-DAGMM additionally records 6.82 seconds of unassigned or harness time. These costs are included in elapsed time and explain why the elapsed difference exceeds the computation difference. Setup and timing-CSV publication are recorded separately outside the round timers. Network communication is not measured in the local simulation.

Figure 9 shows the computation trajectories and their pointwise differences. The additional dashed MNIST curve shows the elapsed-time difference. A positive difference means that STD-DAGMM takes longer, and the horizontal dotted line gives the mean computation difference. The comparison describes the measured workflows under the stated fitting budget. Differences in response handling, diagnostics, and host load prevent attributing the entire gap to detector computation. Removing a free rider also does not remove an honest client's local SGD work.

TABLE V
MEASURED MNIST COMPUTATION AND ELAPSED TIME (S PER ROUND).

Stage	BaitFL	STD-DAGMM
Honest training	34.300 ± 0.937	33.886 ± 0.665
Fabricated responses	0.0185 ± 0.0292	0.0662 ± 0.0115
Client evaluation	0.1613 ± 0.2478	0.000 ± 0.000
Trap construction	0.001621 ± 0.005402	0.000 ± 0.000
Scheduling	0.000147 ± 0.000142	0.000 ± 0.000
Features and preparation	0.1979 ± 0.0501	0.0455 ± 0.0033
Detector fitting	0.000 ± 0.000	15.761 ± 1.246
Decisions and scoring	0.0155 ± 0.0707	0.3346 ± 0.0394
Aggregation	0.0299 ± 0.0131	0.0429 ± 0.0033
Global evaluation	2.196 ± 0.097	2.126 ± 0.042
Research logs and checkpoints	3.995 ± 0.606	3.748 ± 0.538
Unassigned and harness	0.008776 ± 0.002595	6.824 ± 2.332
Total computation	36.921 ± 1.131	52.262 ± 1.492
Elapsed round time	40.924 ± 1.350	62.833 ± 3.493

TABLE VI
ROUND TIMING OVER 100 ROUNDS.

Dataset	Workflow	Mean round time (s)		Full horizon (min)	
		Computation	Elapsed	Computation	Elapsed
MNIST	BaitFL	36.92	40.92	61.53	68.21
	STD-DAGMM	52.26	62.83	87.10	104.72
CIFAR-10	BaitFL	84.85	–	141.41	–
	STD-DAGMM	127.24	–	212.06	–

TABLE VII
BAITFL EXECUTION TIME BY ROUND PHASE.

Dataset	Round phase	Count	Computation (s)	Elapsed (s)	Decision (ms)	Contribution (s/round)
MNIST	Warmup	10	38.03	41.15	0.002	3.80
	Trap response	27	37.40	41.42	0.002	10.10
	Screening round	2	37.46	41.53	373.03	0.75
	Removal round	1	37.52	41.71	395.91	0.38
	Audit round	6	36.41	40.45	68.57	2.18
	Ordinary round	54	36.50	40.65	0.002	19.71
CIFAR-10	Warmup	10	117.76	–	–	11.78
	Trap response	63	82.04	–	–	51.68
	Screening round	6	86.06	–	–	5.16
	Removal round	1	73.50	–	–	0.73
	Audit round	2	76.76	–	–	1.54
	Ordinary round	18	77.51	–	–	13.95

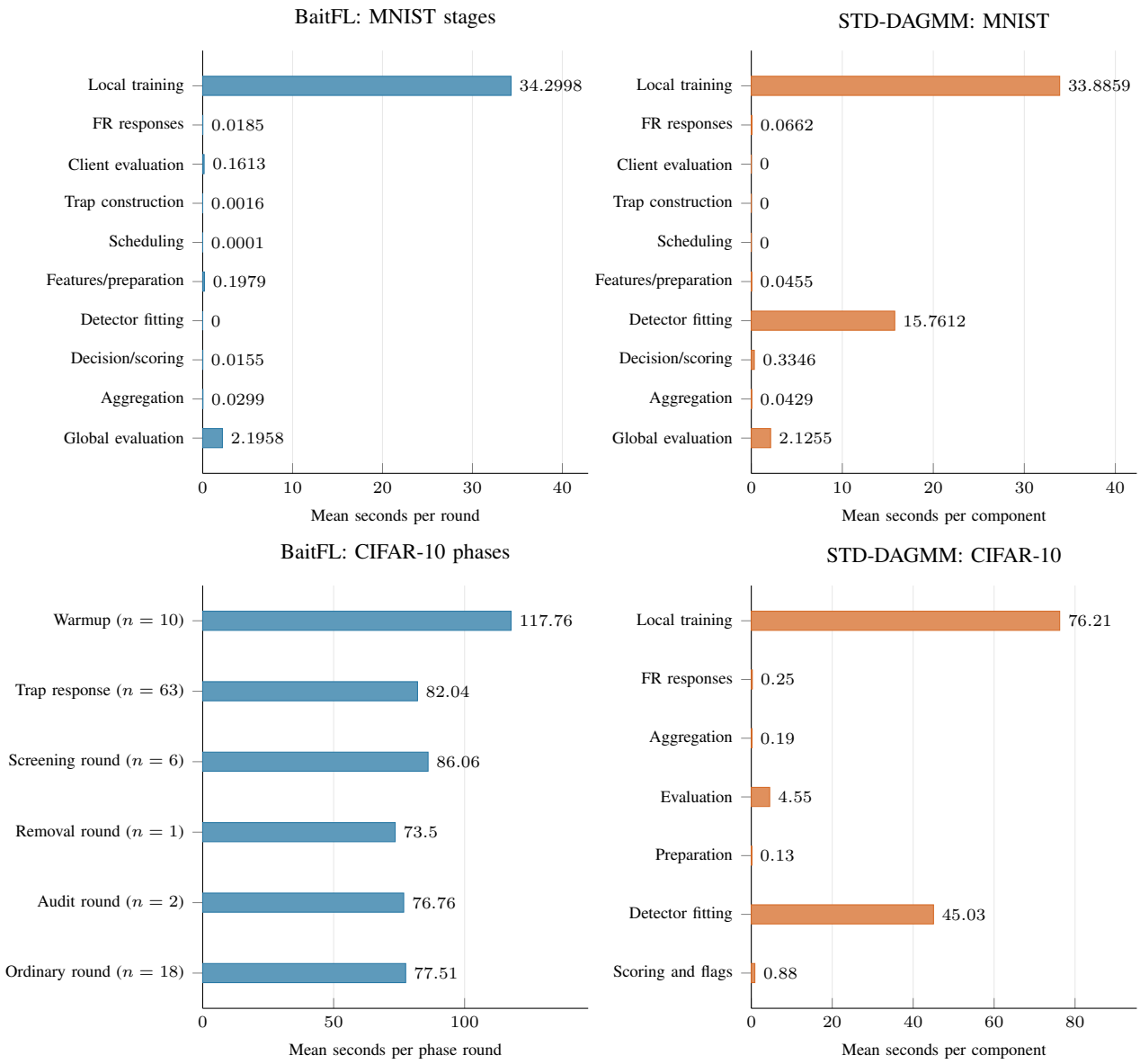


Fig. 7. Measured timing breakdowns.

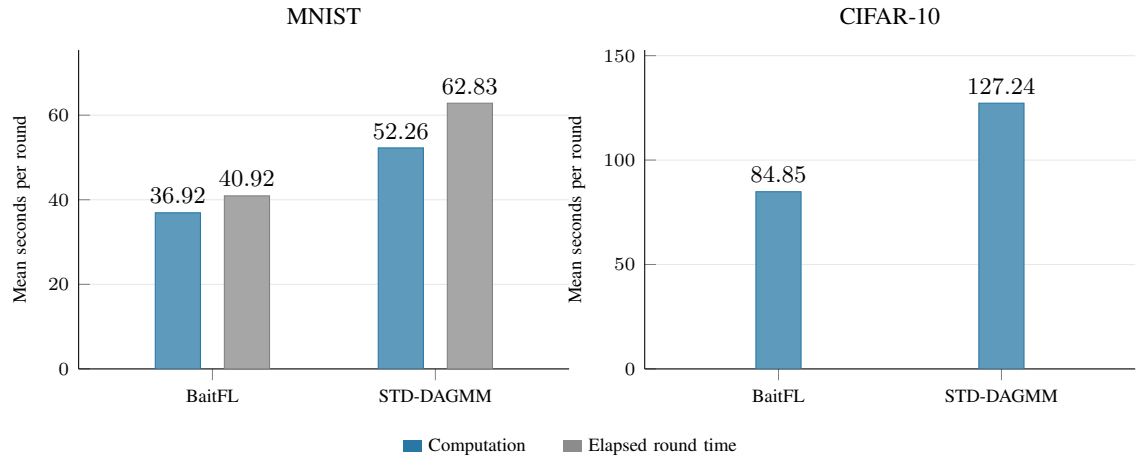


Fig. 8. Average round computation and elapsed time.

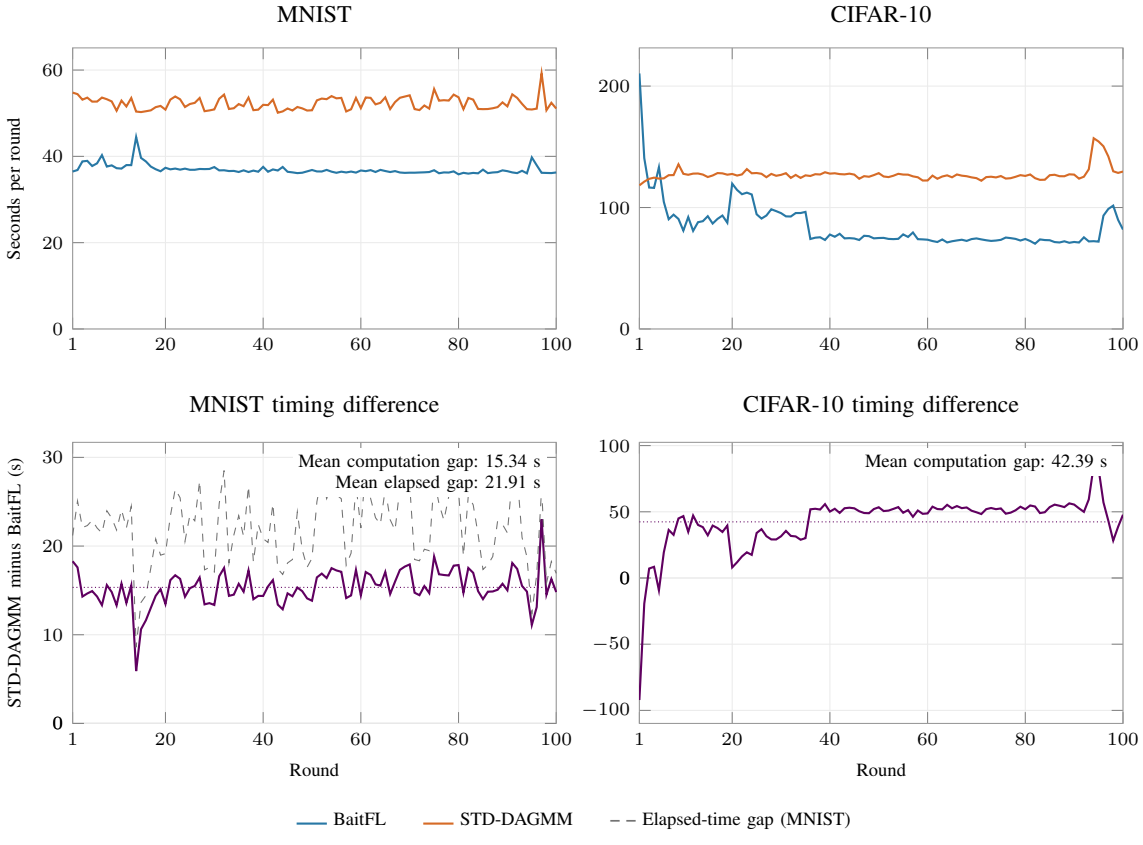


Fig. 9. Round timing trajectories and their measured differences.

VI. Result Analysis

A. Effect of Attack Behavior

FR1 is consistently removed at the earliest allowed round. Exact sendback provides a distinct evidence path, so its success does not validate the statistical separation gate. Near-converged honest zero updates would still require separate examination in other training regimes.

FR2 is removed by round 30 in all settings despite avoiding equality. The observed responses are sufficiently distinct under the configured random update distribution and repeated traps. This is evidence about the tested amplitude, not proof that arbitrary fabricated noise will be detected.

FR3 is the critical heterogeneous-data case. Its current-inclusive history can yield responses that overlap honest update geometry. In non-IID CIFAR-10 at 40% in the representative replication, all 40 free riders receive full norm certificates at rounds 20 and 60; at 30%, all 30 receive them at rounds 40 and 80. Ambiguous intervening cycles do not increment these counts. Retention permits the later fresh certificate to confirm removal. This recorded sequence explains how the memory rule operates; it is not a matched ablation proving that memory alone caused the outcome under a counterfactual training trajectory.

FR4 is removed by round 30 in the representative replication, while additional replications require up to round 90. Its received-trajectory estimate includes traps, which plausibly contributes to separation from genuine optimization. This is an interpretation of the mechanism, not a measurement of the attacker's inference ability. A defense-aware attacker could treat repeated or unusual received states differently and needs a separate evaluation.

B. Dataset, Partition, and Attacker Share

MNIST FR1 to FR3 complete free-rider removal at round 30, while some MNIST FR4 and non-IID CIFAR-10 executions require additional evidence cycles. This shows that neither attack identity alone nor final recall fully describes difficulty. The interaction between honest-data heterogeneity, model state, and fabricated response geometry matters.

In the representative replication, changing the share from 40% to 30% preserves complete detection but delays the difficult CIFAR-10 FR3 case from round 60 to round 80. A lower attacker share therefore does not imply faster discovery. Cohort support and the distribution of the surrounding honest population both change; the server does not use the known simulation share to force a quota or boundary.

The 30% runs also train 70 honest clients rather than 60. This increases genuine local work and helps explain longer simulator round times without implying a more expensive statistical rule. Non-IID client dataset sizes differ, so the increase need not equal the 16.7% change in honest-client count. The slightly higher achieved accuracies at 30% are consistent with more honest training, but no causal effect is established without matched controls.

The matched baseline comparison in Table IV adds a utility perspective. Its largest grouped gain occurs in CIFAR-10 non-IID at 40%, where final accuracy increases by 1.02 percentage points. Figure 6 shows that the benefit varies across fabrication strategies. Removing noncontributing responses can improve the aggregation trajectory, but final accuracy also reflects the honest updates withheld during challenge rounds. The observed changes therefore describe the combined detection and mitigation procedure.

C. Screening Quality Versus Removal Safety

The evaluated procedure achieves high removal precision, but the additional replications show that repeated evidence does not eliminate false positives. The central unresolved issue is that some non-IID honest populations remain almost

entirely candidate-labelled after attackers disappear. This can arise when relative statistical comparisons continue to find weak differences among honest clients. Such labels do not, by themselves, exclude ordinary updates, but they limit the meaning of calling R "safe" or C "malicious." We interpret R as unflagged, C as unresolved, and S as freshly certified suspicion.

Temporary exclusion also remains a real cost. Most honest exclusions are deliberate trap responses, not quarantine. In ordinary successful MNIST runs, 660 honest client-rounds are withheld at 40% and 770 at 30%, corresponding to eleven trap rounds per honest client over the 100-round horizon. The exceptional FR3 follow-up adds further probing. A deployment should measure whether this forgone aggregation delays reaching a target utility, rather than reporting permanent-removal precision alone.

D. What the Current Evidence Does Not Establish

The repeated experiments assess variation across the tested seeds. Wider validation should include honest-only populations, longer training horizons, matched ablations, and profiling under controlled workloads. Client observations within a run are correlated, and complete recall in these experiments does not establish universal detection. Variations of attacker history length, noise amplitude, trap strength, local epoch count, non-IID concentration, and model architecture remain necessary robustness tests. In particular, the influence of CIFAR BatchNorm buffers should be separated from trainable-weight evidence.

VII. Complexity and Lightweight Design

A. Operation Counts and Scaling

Let N be active clients, P floating model coordinates (including floating buffers), L floating tensors, T rounds, B bootstrap replicates and $K = \lceil \sqrt{N} \rceil$ histogram bins. Table VIII summarizes the implementation bounds. A cycle refers to complete coverage by both challenges. The bounds exclude ordinary aggregation, local training, simulated attacker histories, and persistent logging.

The leave-one-out implementation rebuilds $N-1$ peer profiles for every client and trap. Profile medians and distances require $O(N^2 L)$ coordinate work; repeated scalar medians in Python's `statistics.median` add $O(N^2 \log N)$ sorting work. Thus, calling the whole method linear in client count would be incorrect. Histogram binning and bootstrap resampling operate only on scalar scores, not on P-dimensional model vectors. With M complete cycles, the detector work is bounded by

$$W = O(TNP) + O\big(M\left[N^2(L+\log N) + B(N+K)\right]\big). \tag{16}$$

Single-trap audits add corresponding one-view scoring passes; they cannot authorize permanent removal. Dense monitoring supplies more observations but only one preselected A/B pair per client enters each complete-cycle decision.

At $N = 100$, the two-trap profile comparison visits $2N(N-1)L = 158{,}400$ profile coordinates for MNIST and $435{,}600$ for CIFAR-10, before constants for medians and distances. One bootstrap view draws $BN = 100{,}000$ scalar indices; three views process three such sets using reused draws in the current implementation. These counts explain why the evidence layer is compact relative to repeated full-model training, but they do not measure its elapsed time.

B. Payload Sizes, Memory Boundaries and Communication

Table IX derives concrete payload sizes from the implemented architectures for 100 active clients, two challenges, and 1000 bootstrap replicates. Here P includes floating buffers and L counts floating tensors. These are calculated payload sizes rather than measured heap usage. The compact float32 pair payload is $2N(L+3)\cdot 4$ bytes under the stated packed representation; the three scalar slots are an illustrative norm/round/flag allowance, not the actual Python object layout. The two trap models occupy $2P \cdot 4$ floating bytes. Actual dictionaries, lists, Python scalars, allocator overhead, fingerprints and bootstrap temporaries increase memory. For example, bootstrap index gathers and bin-index arrays coexist with resample/count arrays; their two listed sizes must not be relabelled as peak bootstrap RSS.

The decision working state is $O(P + NL + BN + BK)$, plus $O(N)$ counters and fingerprints. In contrast, the research logger accumulates per-round/probe rows, including tensor profiles, so the full simulator's retained evidence can grow as $O(TNL)$ in dense probing. Ordinary aggregation also retains model responses, and the simulator stores attack histories. The measured process RSS in Table III includes these costs; it is not evidence that the entire implementation uses only compact decision-state memory. Streaming logs and aggregation would be deployment optimizations, not features benchmarked here.

No extra local optimization pass or auxiliary classifier is required. Under the existing full-participation request/response schedule, traps replace ordinary models rather than adding a second exchange; challenge responses are discarded from aggregation. The nominal model-payload order remains $O(NP)$ per round, excluding transport/serialization overhead. This does not make communication "free": useful contributions are withheld, so reaching a target accuracy can require more rounds. Network-byte measurements and controlled detector-only profiling are still necessary. We use *lightweight* to mean compact features, no auxiliary learning, and constant-sized per-client evidence memory, not an unmeasured universal speed or energy advantage.

VIII. Security Analysis

A. Decision Inputs and Attack-Label Separation

The detector receives an allowlist of client/trap IDs, raw norms, profile distances, robust scores, and exact-sendback indicators. The simulator's attacker labels and behavior choices are not passed to screening, geometric tests, or the client state machine. Evaluation prevalence determines how many attackers the simulator creates, not how many removals the

TABLE VIII
INCREMENTAL DETECTOR OPERATIONS AND WORKING STATE.

Component	Work	Working state	Practical interpretation
Norm, profile, exact fingerprint	$O(NP)$ per round	$O(P)$ scratch, $O(N)$ fingerprints	Sequential model reads; no auxiliary training or gradients.
Selected A/B summaries	$O(NL)$ per cycle to store	$O(NL)$	Two compact profiles per active client, not full response histories.
Leave-one-out norm/profile scores	$O(N^2(L + \log N))$ per cycle	$O(NL)$ temporary	Repeated peer-profile medians/distances; Python scalar medians sort.
Three histogram/bootstrap views	$O(B(N + K))$ per cycle	$O(BN + BK)$ scratch	Three views add a constant factor; $B = 1000$.
Geometric gates and memory	$O(N)$ selections/quantiles per cycle	$O(N)$ counters	Fixed component count; no learned detector or history-window reconstruction.
Two model challenges	$O(P)$ generation per cycle	$O(P)$	Two shared trap states, not a distinct trap model per client.

TABLE IX
CALCULATED DETECTOR PAYLOAD SIZES.

Model	Parameters	P	L	Two traps (MiB)	Pair payload (KiB)	Index array (MiB)	Counts (MiB)
MNIST	206922	206922	8	1.579	8.594	0.763	0.076
CIFAR-10	620810	621258	22	4.740	19.531	0.763	0.076

detector attempts to achieve. This code boundary reduces direct evaluation leakage; it does not make the development process independent of previously observed results.

B. Confirmation Invariants

Three operational safeguards follow from the state-transition rules. First, statistical removal requires the same component on both selected traps and a second distinct full cycle: one trap, mixed norm/profile evidence, or an ambiguous result cannot independently supply the required confirmation. Second, previously stored evidence does not authorize a new removal without a fresh current certificate. Third, stored counts and weak C evidence cannot renew quarantine; unconfirmed review expires unless fresh strong evidence returns. These are protocol invariants, not probabilistic security bounds.

The guarantees are intentionally narrower than "no false positives." An honest client can satisfy the same geometric gate in two cycles. A clean majority and independent trap draws do not make the resulting statistical certificates independent, because clients retain the same data and related model trajectories.

C. Evasion, Identifiability, and Trap Secrecy

Consider two executions with identical server-observable response histories, public identities, and server randomness, one produced by honest training and the other by fabrication. A detector whose transition function depends only on those observations has the same state trajectory in both executions. Therefore, update-only screening cannot prove genuine computation against every attacker. The proposed method must be evaluated by how well its challenges separate specified behaviors, not as a cryptographic proof of training.

The evaluated attackers are not given trap labels, but several real-world signals could reveal auditing: repeated frozen states, abrupt distributional changes, or communication between clients receiving different models. Moreover, the simulator derives reproducible randomness from configured seeds; it does not establish unpredictability against an adversary that knows those seeds. Deployment would require server-secret randomness and an evaluation of defense-aware response strategies. Even secret randomness alone would not establish indistinguishability of traps from ordinary models.

D. Honest-Client Safety and Long-Term Accumulation

There is no fitted numerical z-score cutoff for statistical removal, but the policy is not parameter-free. Trap scale, histogram binning, bootstrap confidence, cohort support, confirmation count, and review/clearance lengths are design choices. A stable boundary is not a calibrated false-removal probability. At 100 clients the minimum statistical cohort support is ten; an isolated nonzero free rider can therefore escape this cohort gate.

Certificate memory deliberately has no fixed age limit. If an honest client's atypical geometry generates repeated strong evidence without two complete R cycles, it can eventually be removed. The profile gate's concentration alternative and persistent candidate populations make this a material long-horizon risk. The honest removals observed in the repeated experiments show that false evidence can lead to exclusion within 100 rounds. Longer honest-only and heterogeneous-population experiments are needed before making deployment safety claims.

E. Privacy, Collusion, and Availability

The protocol is not directly compatible with standard secure aggregation and does not provide differential privacy [11]. Avoiding client-reported loss and raw data access reduces required inputs but is not a privacy proof: identifiable returned models may still leak information. Colluding clients could

share a genuinely trained response or infer the trap schedule, and Sybil identities could distort the assumed honest-majority reference. Neither attack is evaluated.

All active clients and complete A/B coverage are required for a certifying cycle. Missing/nonfinite measurements cannot certify, and the current simulator aborts on a nonfinite client response rather than implementing a complete availability defense. Dropout tolerance, malicious-message handling, restart integrity, authentication, and denial-of-service resistance require additional engineering and experiments. Checkpoint recovery preserves computation and evidence state after interruption; it does not create new evidence of honesty.

F. Computational and Communication Exposure

The bounds in Section VII expose quadratic-in-population leave-one-out work and transient bootstrap memory. A large population or model can make auditing expensive even without learned detector training. Deliberately withheld trap responses and repeated honest follow-up are an availability and utility cost, not merely an implementation detail. The MNIST operation timers in Section V-H quantify measured feature and decision work. Communication bytes, time to target utility, and energy remain unmeasured.

IX. Conclusion

This study evaluates a lightweight, server-observable challenge approach to free-rider screening, with strict norm separation, component-specific evidence memory, and bounded review. Evaluation across multiple random seeds on MNIST and CIFAR-10 achieves 100.00% recall and pooled precision of 99.91%. Most executions complete detection by round 30, with more difficult cases requiring up to round 90. The three honest removals show that repeated confirmation does not eliminate false positives. The results show useful separation under the evaluated training and attack configurations, while the candidate and quarantine results show why final recall and precision alone are insufficient. Matched comparisons with unprotected federated averaging improve mean final accuracy by 0.10 percentage points on MNIST and 0.64 on CIFAR-10. The measured STD-DAGMM comparison also shows lower average round computation under the stated five-epoch fitting budget. The instrumented MNIST comparison measures 36.92 versus 52.26 seconds per round and separates the cost of response checks from logging and checkpoint publication.

Further validation should include broader client populations, honest-only training, and tests of adaptive/mixed attacks and longer horizons. Matched ablations and detector-only profiling should distinguish the contributions of traps, geometric gates, and memory from ordinary training and simulator costs. The method is a promising empirical screening mechanism, not a universal verifier of local computation.

Appendix A
Detection and Utility by Attack

Tables X and XI retain detailed outcomes for each dataset, partition, free-rider proportion, and attack. Entries are means across seeds, including the counts and the complete-detection round t_{all}. Here C_H is the final honest-candidate count, Q_H is the number of distinct honest clients ever quarantined, and q_H is the number of honest quarantine client rounds. Full records for the repeated experiments, including the honest removals, are provided in `generated/multiseed/per_run.csv`. Group means and accuracy standard deviations are retained in `generated/multiseed/table1_means.csv`.

Appendix B
Supporting Trajectories and Baseline Replication

Figure 10 retains the representative cumulative detection trajectories, including the delayed non-IID CIFAR-10 FR3 cases. Figure 11 restores the full training horizon and shared 0 to 100% scale for interpreting the matched accuracy comparison. Table XII reports the additional completed CIFAR-10 non-IID replication at 40%. It uses the same accuracy and timing differences as Table IV. These executions are retained separately because the additional baseline replication does not cover the complete experimental grid.

Appendix C
Reproducibility and Interpretation Notes

Tables I, II, III, X, and XI are generated by `build_multiseed_table.ps1`. It validates completion markers, configuration coverage, training rounds, initial A/B coverage, removal identities and timing, summary counts, honest-client exclusion records, final accuracy, detector policy, and six recorded implementation hashes across all replications. Seed identifiers and input hashes are retained in `generated/multiseed/provenance.json`, with individual outcomes and grouped means in the accompanying CSV files. These records include failed detections and honest removals rather than selecting only favorable outcomes.

The score diagnostics use the representative replication. Their input hashes are recorded in `generated/provenance.json`, and the six matching source files are copied to `evidence/source/`. These hashes describe the recorded implementation and do not attest the complete historical environment. The score exports retain raw paired coordinates and per-class means, medians, quartiles, finite counts and undefined counts for every complete evidence cycle. The architecture hash is recorded in `generated/visual_provenance.json`.

The score diagnostics show individual executions. Figure 3 is generated by `build_norm_figure.ps1`, which checks the plotted populations and paired norm-certificate counts against the recorded responses and decisions. The workflow timing tables retain their separately measured comparison cases. `build_baseline_comparison.ps1` validates the completed no-defense executions, matching configurations, complete participation, absence of filtering, and agreement during warmup. Paired outcomes, trajectories, and input hashes are retained under `generated/comparison/`. `build_review_figures.ps1` generates the combined score panels, baseline comparisons, and timing figures. Phase counts cover all 100 rounds, and the STD-DAGMM stage means sum

TABLE X
DETECTION AND UTILITY BY ATTACK ON MNIST.

FR (%)	Partition	Attack	TP	FP	FN	t_{all}	Accuracy (%)	C_H	Q_H	q_H
30	IID	FR1	30.00	0.00	0.00	30.00	99.01	1.00	0.00	0.00
30	IID	FR2	30.00	0.00	0.00	30.00	99.01	4.00	0.00	0.00
30	IID	FR3	30.00	0.00	0.00	30.00	99.02	6.33	0.00	0.00
30	IID	FR4	30.00	0.00	0.00	30.00	98.90	5.33	0.00	0.00
30	Non-IID	FR1	30.00	0.00	0.00	30.00	98.92	5.00	0.00	0.00
30	Non-IID	FR2	30.00	0.00	0.00	30.00	98.93	4.67	0.00	0.00
30	Non-IID	FR3	30.00	0.00	0.00	30.00	98.93	41.67	1.33	26.67
30	Non-IID	FR4	30.00	0.00	0.00	40.00	98.91	40.67	0.00	0.00
40	IID	FR1	40.00	0.00	0.00	30.00	98.93	0.67	0.00	0.00
40	IID	FR2	40.00	0.00	0.00	30.00	98.94	5.33	0.00	0.00
40	IID	FR3	40.00	0.00	0.00	30.00	98.92	4.33	0.00	0.00
40	IID	FR4	40.00	0.00	0.00	36.67	98.83	16.33	0.00	0.00
40	Non-IID	FR1	40.00	0.00	0.00	30.00	98.91	15.00	0.00	0.00
40	Non-IID	FR2	40.00	0.00	0.00	30.00	98.91	6.33	0.00	0.00
40	Non-IID	FR3	40.00	0.67	0.00	30.00	98.90	37.00	3.33	60.00
40	Non-IID	FR4	40.00	0.33	0.00	60.00	98.88	35.00	1.00	20.00

TABLE XI
DETECTION AND UTILITY BY ATTACK ON CIFAR-10.

FR (%)	Partition	Attack	TP	FP	FN	t_{all}	Accuracy (%)	C_H	Q_H	q_H
30	IID	FR1	30.00	0.00	0.00	30.00	77.84	2.33	0.00	0.00
30	IID	FR2	30.00	0.00	0.00	30.00	77.84	0.00	0.00	0.00
30	IID	FR3	30.00	0.00	0.00	30.00	77.76	2.33	0.00	0.00
30	IID	FR4	30.00	0.00	0.00	30.00	77.77	5.67	0.00	0.00
30	Non-IID	FR1	30.00	0.00	0.00	30.00	75.94	17.00	0.00	0.00
30	Non-IID	FR2	30.00	0.00	0.00	30.00	75.86	5.00	0.00	0.00
30	Non-IID	FR3	30.00	0.00	0.00	66.67	75.77	42.67	0.67	13.33
30	Non-IID	FR4	30.00	0.00	0.00	33.33	75.78	27.33	0.67	13.33
40	IID	FR1	40.00	0.00	0.00	30.00	77.07	0.33	0.00	0.00
40	IID	FR2	40.00	0.00	0.00	30.00	77.13	0.33	0.00	0.00
40	IID	FR3	40.00	0.00	0.00	30.00	76.92	4.00	0.00	0.00
40	IID	FR4	40.00	0.00	0.00	30.00	76.97	4.33	0.00	0.00
40	Non-IID	FR1	40.00	0.00	0.00	30.00	74.70	6.00	0.00	0.00
40	Non-IID	FR2	40.00	0.00	0.00	30.00	74.88	3.67	0.00	0.00
40	Non-IID	FR3	40.00	0.00	0.00	70.00	74.67	43.00	3.67	73.33
40	Non-IID	FR4	40.00	0.00	0.00	36.67	74.51	14.00	0.00	0.00

TABLE XII
ADDITIONAL MATCHED CIFAR-10 NON-IID COMPARISONS AT 40% FREE RIDERS.

	Final accuracy (%)			Mean round time (s)		
Attack	No defense	BaitFL	Δ (pp)	No defense	BaitFL	Δ (%)
FR2	74.49	75.58	1.09	71.04	74.91	5.45
FR3	72.35	75.18	2.83	82.71	81.35	-1.64
FR4	75.41	75.21	-0.20	75.77	79.54	4.96

to its reported computation time. The instrumented MNIST evidence is preserved under `evidence/timing_mnist_20261103/`. Its import manifest records the study commit, original Git object identifiers, and SHA-256 hashes. `build_instrumented_mnist_timing.ps1` validates completion, matching settings, instrumentation sources, exclusive timer sums, elapsed-time accounting, and phase coverage. Nested joint-training and mixture timers are checked as parts of fitting and are not added again to total computation. The accuracy trajectory agrees with the corresponding study execution in all 100 rounds. Compile `paper_v7.tex` with PDFLaTeX and BibTeX, or use the accompanying `build.ps1`. Previous paper versions and all experiment logs remain unchanged.

Interrupted runs resume their last published checkpoint with loader/RNG state, attack histories, selected probe positions, quarantine and confirmation memory. The runtime table reports logged execution measures rather than powered-off calendar waiting. None of the reported resource values should be relabelled as detector-only overhead or interpreted as evidence of privacy/security guarantees.

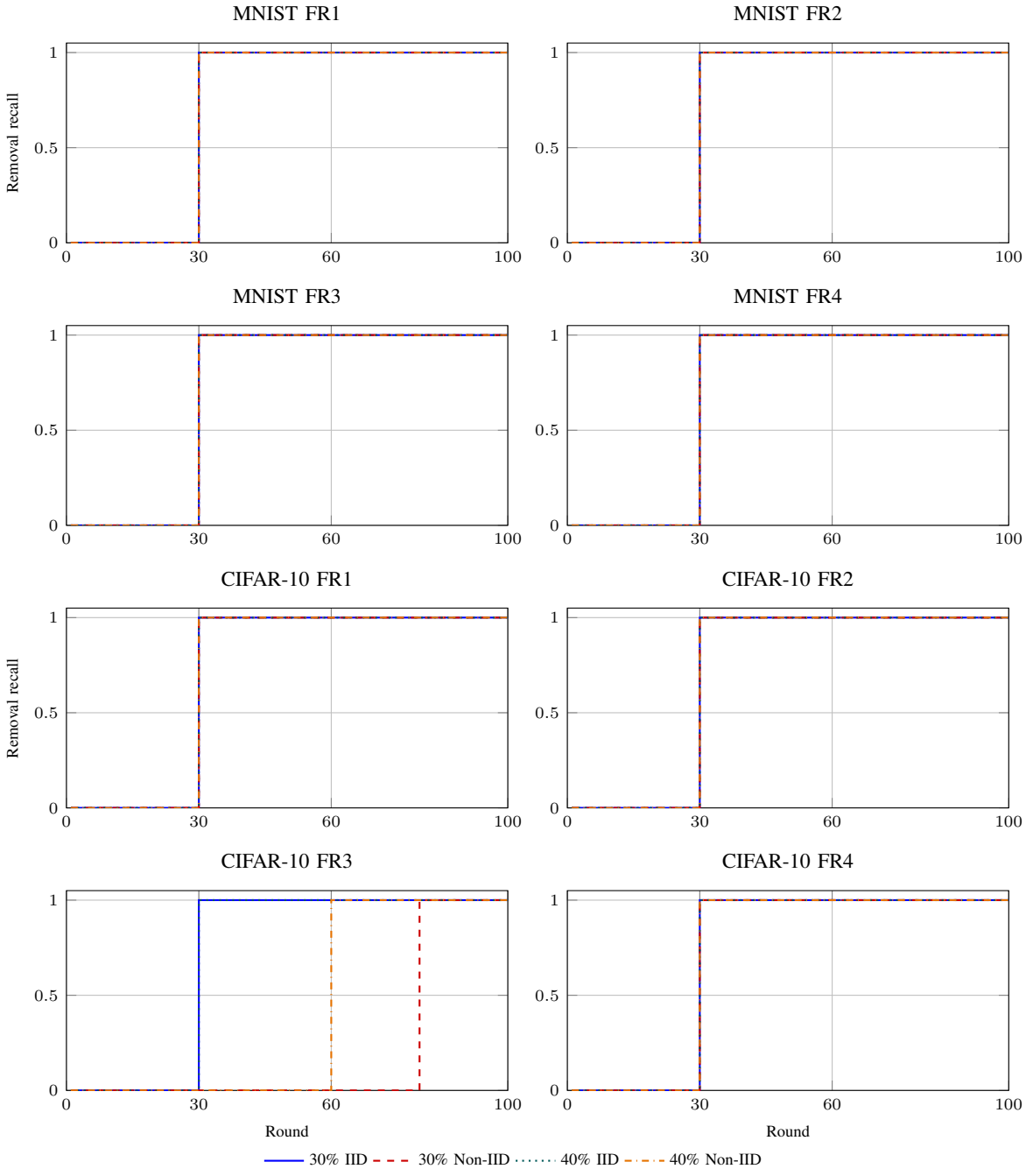


Fig. 10. Free-rider removal over training.

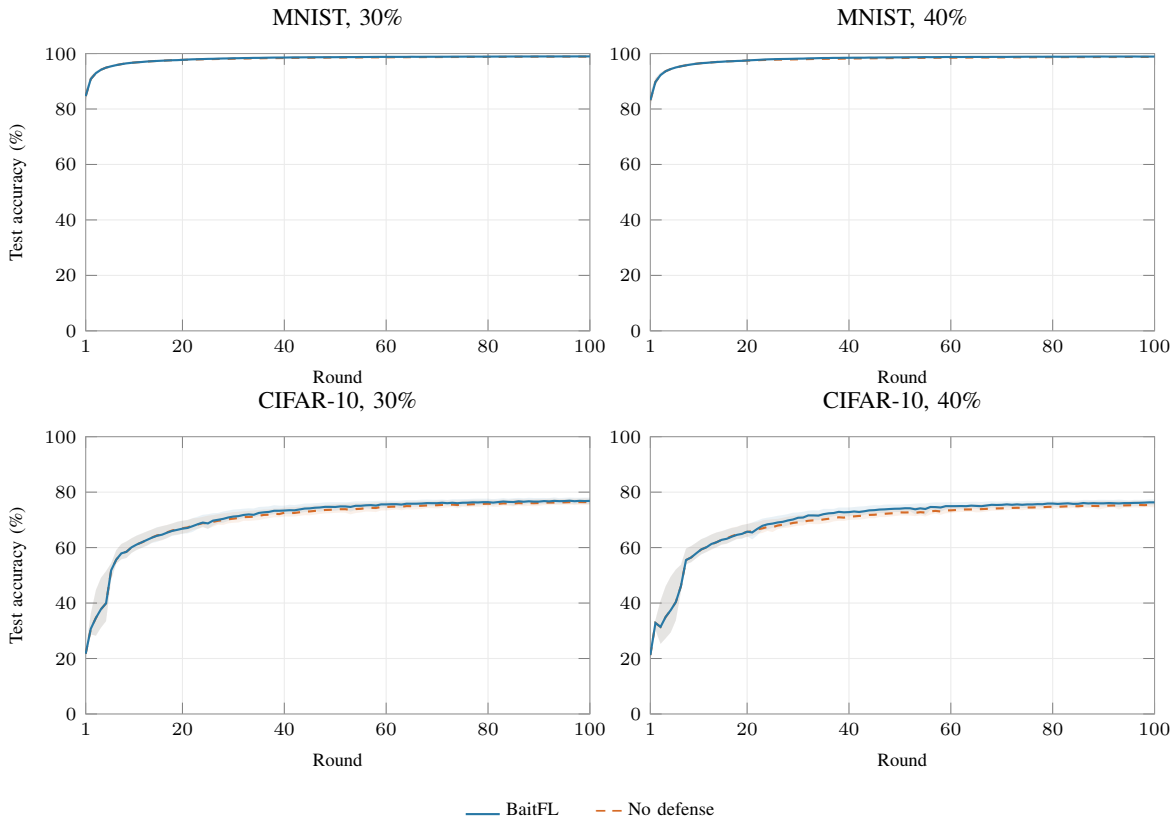


Fig. 11. Full-horizon accuracy with and without BaitFL.

References

[1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 54. PMLR, 2017, pp. 1273–1282. [Online]. Available: https://proceedings.mlr.press/v54/mcmahan17a.html

[2] N. Rieke, J. Hancox, W. Li *et al.*, "The future of digital health with federated learning," *npj Digital Medicine*, vol. 3, 2020, art. no. 119. [Online]. Available: https://www.nature.com/articles/s41746-020-00323-1

[3] P. Kairouz, H. B. McMahan, B. Avent *et al.*, "Advances and open problems in federated learning," *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021. [Online]. Available: https://arxiv.org/abs/1912.04977

[4] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, "How to backdoor federated learning," in *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 108. PMLR, 2020, pp. 2938–2948. [Online]. Available: https://proceedings.mlr.press/v108/bagdasaryan20a.html

[5] J. Lin, M. Du, and J. Liu, "Free-riders in federated learning: Attacks and defenses," arXiv preprint arXiv:1911.12560, 2019. [Online]. Available: https://arxiv.org/abs/1911.12560

[6] Y. Fraboni, R. Vidal, and M. Lorenzi, "Free-rider attacks on model aggregation in federated learning," in *Proceedings of the 24th International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 130. PMLR, 2021, pp. 1846–1854. [Online]. Available: https://proceedings.mlr.press/v130/fraboni21a.html

[7] J. Wang, X. Chang, J. Mišić, V. B. Mišić, and Y. Wang, "PASS: A parameter audit-based secure and fair federated learning scheme against free-rider attack," *IEEE Internet of Things Journal*, vol. 11, no. 1, pp. 1374–1384, 2024. [Online]. Available: https://arxiv.org/abs/2207.07292

[8] P. G. Recasens, Á. Horváth, A. Gutierrez-Torre, J. Torres, J. L. Berral, and B. Pejó, "FRIDA: Free-rider detection using privacy attacks," *Journal of Information Security and Applications*, vol. 97, 2026, art. no. 104357. [Online]. Available: https://arxiv.org/abs/2410.05020

[9] Z. Zhu, J. Shu, X. Zou, and X. Jia, "Advanced free-rider attacks in federated learning," in *NeurIPS Workshop on New Frontiers in Federated Learning*, 2021. [Online]. Available: https://neurips2021workshopfl.github.io/NFFL-2021/papers/2021/Zhu2021.pdf

[10] B. Wang, H. Li, X. Liu, and Y. Guo, "FRAD: Free-rider attacks detection mechanism for federated learning in AIoT," *IEEE Internet of Things Journal*, vol. 11, no. 3, pp. 4377–4388, 2024. [Online]. Available: https://doi.org/10.1109/JIOT.2023.3298606

[11] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, "Practical secure aggregation for privacy-preserving machine learning," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017, pp. 1175–1191. [Online]. Available: https://research.google/pubs/practical-secure-aggregation-for-privacy-preserving-machine-learning/

[12] N. Otsu, "A threshold selection method from gray-level histograms," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979. [Online]. Available: https://doi.org/10.1109/TSMC.1979.4310076

[13] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979. [Online]. Available: https://doi.org/10.1214/aos/1176344552

[14] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. [Online]. Available: https://doi.org/10.1109/5.726791

[15] A. Krizhevsky, "Learning multiple layers of features from tiny images," University of Toronto, Tech. Rep., 2009. [Online]. Available: https://www.cs.toronto.edu/~kriz/cifar.html